Engineering Analysis with Boundary Elements 176 (2025) 106200

Contents lists available at ScienceDirect

Engineering Analysis with Boundary Elements

journal homepage: www.elsevier.com/locate/enganabound

PINNs-MPF: A Physics-Informed Neural Network framework for Multi-Phase-Field simulation of interface dynamics

Seifallah Elfetni [a], Reza Darvishi Kamachali [a,b,*]

[a] Federal Institute for Materials Research and Testing (BAM), 12205 Berlin, Germany
[b] Institute of Materials Physics, University of Münster, 48149 Münster, Germany

ARTICLE INFO

Dataset link: https://github.com/SFETNI/PINNs_MPF

Keywords:
PINNs
Phase-field method
Microstructure evolution
Parallel training
Neural networks
Machine learning

ABSTRACT

We present PINNs-MPF framework, an application of Physics-Informed Neural Networks (PINNs) to handle Multi-Phase-Field (MPF) simulations of microstructure evolution. A combination of optimization techniques within PINNs and in direct relation to MPF method are extended and adapted. The numerical resolution is realized through a multi-variable time-series problem by using fully discrete resolution. Within each interval, space, time, and phases/grains are treated separately, constituting discrete subdomains. PINNs-MPF is equipped with an extended multi-networking (parallelization) concept to subdivide the simulation domain into multiple batches, with each batch associated with an independent NN trained to predict the solution. To ensure continuity across the spatio-temporal-phasic subdomains, a Master NN efficiently is to handle interactions among the multiple networks and facilitates the transfer of learning. A pyramidal training approach is proposed to the PINN community as a dual-impact method: to facilitate the initialization of training when dealing with multiple networks, and to unify the solution through an extended transfer of learning. Furthermore, a comprehensive approach is adopted to specifically focus the attention on the interfacial regions through a dynamic meshing process, significantly simplifying the tuning of hyper-parameters, serving as a key concept for addressing MPF problems using machine learning. We perform a set of systematic simulations that benchmark foundational aspects of MPF simulations, i.e., the curvature-driven dynamics of a diffuse interface, in the presence and absence of an external driving force, and the evolution and equilibrium of a triple junction. The proposed PINNs-MPF framework successfully reproduces benchmark tests with high fidelity and Mean Squared Error (MSE) loss values ranging from 10^{-6} to 10^{-4} compared to ground truth solutions.

1. Introduction

Today's microstructure modeling has grown into an extensive branch of materials research, on the one hand, revealing the multi-physics of the concurrently evolving microstructural constituents across scales and, on the other hand, bridging these with the process, property and performance of materials. Despite remarkable advances over the last two decades, the rapidly growing complexities of materials' chemistry and processing are constantly surpassing our actual microstructure modeling capacity, thus, decelerating materials innovation. A potential solution for this growing challenge is to utilize the recent developments in machine learning (ML) and deep learning (DL) rapidly emerging in various areas of materials modeling [1–6]. The recently introduced Physics-Informed Neural Networks (PINNs) are showing promising capabilities in addressing physical phenomena [7]. By enabling nonlinear approximations of solutions for Partial Differential Equations (PDEs), the PINNs counterpart of the conventional approaches, such as phase-field, fluid dynamics and finite-element methods, would allow for a mesh-free modeling framework, capable of transfer of the learning and without the necessity of local approximations or simplifications in the underlying physics [7–9]. These promote new perspectives in microstructure modeling, motivating us to pursue the current line of studies.

First applications of PINNs to solve 1D PDEs (e.g., the heat equation, wave equation and Burgers' equation) were quite successful, demonstrating its simplicity and efficiency [10–12]. Since then, deeper applications have been increasingly targeted in various areas including heat transfer [13,14], solid mechanics [8,15–22], electromagnetic [8,9,23], turbulence modeling [24,25], electro-chemistry [26,27] and energy [28,29]. However, as the literature on resolving PDEs using PINNs has grown, several challenges have been revealed, highlighting the necessity for optimization techniques tailored to specific physical contexts. Jagtap et al. [30] proposed Extended Physics-Informed Neural

* Corresponding author.
E-mail addresses: seifallah.elfetni@bam.de (S. Elfetni), reza.kamachali@bam.de (R.D. Kamachali).

https://doi.org/10.1016/j.enganabound.2025.106200
Received 13 November 2024; Received in revised form 31 January 2025; Accepted 2 March 2025
Available online 1 April 2025

Networks (XPINNs) to address moving boundaries. This method subdivides the domain into different mini-batches to locally compute the solution using distinct neural networks. XPINNs were tested on the one-dimensional viscous Burgers equation, employing various optimization techniques such as subdomain decomposition (in time and space), adaptive activation functions, independent hyper-parameter adjustment for each network, and a specific data processing method to characterize the average behavior of predictions along the subdomain interfaces. Zhang and Li [31] propose the Deep Interface Alternation Method (DIAM), a domain decomposition and deep learning approach for solving complex elliptic interface problems. DIAM converts interface issues into continuous subproblems, using separate neural networks for each subdomain and sharing interface data through loss functions to achieve accurate solutions. Kharazmi et al. [32] introduced hp-Variational Physics-Informed Neural Networks (hp-VPINNs), utilizing domain decomposition and h-refinement with dynamic mesh resolution adjustment to target the advection–diffusion equation (1D) and Poisson equation (1D-2D). Meng et al. [33] propose a Parareal PINNs (PPINN) framework for solving time-dependent PDEs using domain decomposition. The examples include ordinary differential equations (ODEs) and a 2D nonlinear diffusion-reaction equation. They employ various optimization techniques like alternating between Adam and L-BFGS optimizers, Quasi-Monte Carlo sampling for stochastic ODEs, and principally a parallel-in-time training approach with a serial update at the subdomain interfaces using the fine PINN solutions as a parallel prediction–correction process. Penwarden et al. [34] further enhanced these techniques by introducing stacked-decomposition inspired by causality literature (time-causality enforcement methods [35,36]), window-sweeping, alternance of optimizers and transfer of learning methods proposed to overcome training challenges, respect causality, and improve scalability by limiting computation per optimization iteration. The authors consider the 1D convection equation, 1D Allen-Cahn equation and 1D Korteweg–de Vries. While the reported methods have demonstrated promising results, further testing and extension are necessary to address more complex scenarios, such as two-dimensional configurations involving systems of equations and three-dimensional domains. These scenarios introduce additional challenges that require innovative approaches and optimization techniques.

A few applications of PINNs to handle diffuse interfaces were recently presented, especially to study evolution of multiple interacting phases or components [37–43]. Zheng et al. [44] have developed a physics-constrained neural network (PCNN) to predict the sequential motions of flow simulations. Due to the challenging nature of multi-phase problems, the applied approach was based on the satisfaction of the mass conversation constraint when encoding the observation of the recurrent NN. A conservative boundedness mapping algorithm (MCBOM) was then called to correct the phase prediction, reported to be efficient for handling sequential motions in some testing benchmarks in 2D, e.g., the evolution of three phases in the shear layer and dam break problems. Haghighat et al. [39] proposed a dimensionless form of the PDEs combined with a sequential training strategy based on stress-split algorithms and multi-networking, applied for solving several problems in poroelasticity. It was found that both the size and structure of the NN, along with the hyperparameters of the optimizer, such as the learning rate, can significantly influence the quality of PINN solutions. Amini et al. [41] introduced inverse modeling of nonisothermal multiphase poromechanics using PINNs, successfully applying the method to both single- and multi-phase flow in porous media.

During these remarkable advances, several challenges were also spotted in the field that remain to be addressed. A primary concern with PINNs involves the gradient pathology in the total loss function, due to larger gradients in higher-derivative terms, affecting the accuracy of the predictions [39]. Furthermore, ensuring systematic convergence of training becomes difficult, especially with the high computational cost of multi-networking. The use of a multi-networking approach can generate differing architectures or training data among separate NNs, which leads to imperfect alignment and thus discontinuities or artifacts along boundaries. Adjusting the size of training sets, including initial conditions, collocation points, and boundary conditions, was shown to help resolve this problem, but at the same time, this adds to the complexity and computational costs [7,45]. The assembly and transfer of collective learning from multiple networks (transferring the learning) is another major challenge in PINNs. Here tuning hyper-parameters, even with a single NN, remains challenging: Adaptive weighting, while introduced to address feature dependencies, reflects non-systematic learning of solution patterns [26]. In addition to these, the profound sensitivity of the solution to the hyper-parameters, encompassing variables such as the number of neurons, network layers, learning rate, and the composition of training datasets, emerges as a major issue that requires more attention when dealing with PINNs [26,46]. Regarding, Multi-phase Field (MPF) equations, they involve a coupled system of nonlinear PDEs with multiple phases and several parameters. The presence of higher-order derivative terms and a non-convex potential term further contributes to additional non-linearities and the emergence of several local minima. Additionally, the generated solutions often require correction algorithms to satisfy conservation laws, rendering classical PINNs unable to make accurate predictions. Therefore, incorporating conservative correction algorithms and adequate optimization methods becomes necessary [47].

Multi-phase-field (MPF) methods for microstructure modeling have proven to be powerful tools for capturing intricate multi-physics phenomena in multi-phase, multi-component materials [48,49]. Being based on a generalizable free energy functional, a key feature of the MPF framework is its capacity to be flexibly expanded to different degrees of complexity which can be specific to the physics of the problem at hand. A representative demonstration of this flexibility has been explored in dealing with polycrystalline microstructures: Starting from curvature-driven interface kinetics, the MPF framework has gradually expanded to investigate complex phenomena such as phase transition, precipitation, recrystallization, and phase transformation as well as various chemo-mechanically coupled phenomena [50–57]. Despite successful implementations and applications [58,59], an intrinsic challenge of the MPF approach is that with any further advancement and raising scientific questions, immediate needs for extensive programming and optimization efforts are required, which often spread to deep levels of redefining key parameters and functions as well as memory- and storage-structures in the code. These originate from the fact that any update to the model necessitates changes to the underlying free energy functional and corresponding equations of motion. As a result, it becomes difficult to keep the software updates up with the pace of emerging topics in materials research, for instance, the unanswered questions regarding non-equilibrium microstructure evolution in additively manufactured materials [60,61] or the complex diffusion-interface couplings in novel high-entropy alloys [62,63].

In this work, we present a comprehensive framework, termed PINNs-MPF, specifically designed to tackle the challenges of solving MPF equations using PINNs. By identifying the limitations of existing PINN methods when applied to MPF equations and recognizing the additional complexities introduced by these nonlinear, convex, and constrained systems, we propose a unified approach that combines and extends relevant PINN techniques with MPF method strategies. The applications of the MPF method are intrinsically complex, addressing large-scale multi-phase, multi-component systems. Yet, in order to have a realistic take on developing the PINNs-MPF framework, it is necessary to decompose the method into parts, first addressing the foundational aspects of curvature-driven dynamics of interfaces and junctions. To this aim, we have to focus on four well-defined scenarios, as detailed in the following. This strategy is inspired by the core development of the OpenPhase software for MPF simulations [64], which has grown to a successful and sustainable software package.

The current framework introduces key novelties through its optimization strategies: (i) A extended domain decomposition approach

is implemented that uses a central coordinating network (referred to as the "Master") to manage tasks and ensure continuity among subnetworks (referred to as the "workers") during full parallel training in space, time and phases. (ii) An extreme mesh refinement approach, with a distinct focus on the phase-field interfacial zones, is applied. (iii) We simplify and reduce the hyper-parameters as elaborated in the following. (iv) A synchronization step to ensure efficient interactions between the evolving phase-fields is applied. Finally, (v) a pyramidal training approach is applied as a robust alternative to adaptive weighting and an efficient solution for transferring learning of multiple-to-multiple and single-to-multiple networks. Inspired by the MPF benchmark philosophy developed in the OpenPhase context [59, 64], we test our PINNs across four core applications that built the MPF concept with a progressive complexity: First, we model the motion of a diffuse interface (propagation of a stable phase-field profile), second and third, we study the curvature-driven shrinkage of a grain in the presence and absence of a driving force, and fourth, we investigate the evolution of a triple junction that demonstrates the core principles of a multi-phase evolution. The results are compared against the outputs from OpenPhase and discussed in the context of our hyper-parameters and concepts implemented within our PINNs framework.

2. The PINNs-MPF framework

The MPF temporal evolution is governed by the set of PDEs as follows:

$$\dot{\phi}_\alpha = \frac{\mu\sigma}{N}\sum_{\beta=1}^{N}\left(\nabla^2\phi_\alpha - \nabla^2\phi_\beta + \frac{\pi^2}{2\eta^2}(\phi_\alpha - \phi_\beta)\right) + \frac{\mu}{N}\sum_{\beta=1}^{N}\Delta G\left(\phi_\alpha, \phi_\beta\right), \quad (1)$$

with phase-fields $\phi_\alpha \in [0,1]$, μ the interface mobility, σ the interface energy and η the interface width. The pairwise term ΔG counts for any additional driving force (e.g. chemical, elastic, etc.) influencing the interface dynamics. At every point in space and time, the summation of phase-fields/order parameter reads:

$$\sum_{\alpha=1}^{N}\phi_\alpha = 1, \quad (2)$$

with N the number of phase-fields present. Further details of the MPF model are given in the Methods Section 3. The PINNs-MPF framework is targeted to resolve Eqs. (1) and (2) in time and space and for various initializations. In doing so, PINNs aim to minimize a loss function $\mathcal{L}(\theta)$ with respect to the hyper-parameters θ. With the MPF model in hand, this is composed of several terms:

$$\begin{aligned}\theta^* &= \arg\min_\theta\left[\mathcal{L}(\theta)\right]\\ &= \arg\min_\theta\left[\lambda_{\text{pde}}\mathcal{L}_{\text{PDE}}(\theta) + \lambda_{\text{bc}}\mathcal{L}_{\text{BC}}(\theta) + \lambda_{\text{ic}}\mathcal{L}_{\text{IC}}(\theta) + \lambda_{\sum\phi}\mathcal{L}_{\sum\phi}(\theta)\right]\end{aligned} \quad (3)$$

where $\mathcal{L}_{\mathcal{PDE}}$ is the loss associated with enforcing the governing physics through PDEs:

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_r}\sum_{i=1}^{N_r}\left[\boldsymbol{r}\left(\boldsymbol{x}_r^i, t_r^i, \theta\right)\right]^2 \quad (4)$$

in which, $\boldsymbol{r}$ represents the residual of the PDEs or the physical loss (Eq. (1)) and N_r the number of collocation points. $\mathcal{L}_{\mathcal{BC}}$ is the loss associated with enforcing boundary conditions (BC) on the simulation domain:

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N_{\text{BC}}}\sum_{i=1}^{N_{\text{BC}}}\left[\boldsymbol{u}\left(\boldsymbol{x}_{\text{BC}}^i, t_{\text{BC}}^i, \theta\right) - g_{\text{BC}}^i\right]^2 \quad (5)$$

with $\boldsymbol{u}$ representing the approximated solution at boundaries $\boldsymbol{x}_{\mathcal{B}}^i$ and time $t_{\mathcal{B}}^i$ using the neural network with parameters θ and, $g_{\mathcal{B}}^i$ is the prescribed or observed value at the corresponding boundary and time. $\mathcal{L}_{\text{IC}}$ is the loss associated with enforcing initial conditions (IC), i.e., the spatial configuration of the phase-fields:

$$\mathcal{L}_{\text{IC}}(\theta) = \frac{1}{N_{\text{IC}}}\sum_{i=1}^{N_{\text{IC}}}\left[\boldsymbol{u}\left(\boldsymbol{x}_{\text{IC}}^i, 0, \theta\right) - h_{\text{IC}}^i\right]^2 \quad (6)$$

that compares the approximated solution at initial state $\boldsymbol{x}_{\text{IC}}^i$ and time $t = 0$ against the prescribed or observed value at the corresponding (initial) state h_{IC}^i. Finally, $\mathcal{L}_{\sum\phi}$ is the loss associated with the constraint Eq. (2) of the phase summation:

$$\mathcal{L}_{\sum\phi}(\theta) = \left|\sum_{\alpha=1}^{N}\phi_\alpha\left(\boldsymbol{x}, t, \theta\right) - 1\right|^2 \quad (7)$$

where N is the total number of phases in the given point. Unlike the other losses which are computed over a subpart of the domain, the loss over the sum constraint is computed for the entire simulation domain.

The coefficients λ_{PDE}, λ_{BC}, λ_{IC} and $\lambda_{\sum\phi}$ in Eq. (3) are weight factors associated with different loss terms, to balance the importance of each term. The objective of Eq. (3) is to find the optimal parameters θ^* that minimize the whole loss function, resulting in *learned* neural networks that are then capable of approximating the MPF solutions of a problem with the given initialization and boundary conditions. We note that since the MPF method is based on a free energy functional, reinforcing terms to account for the energy dissipation [65] could be directly added to the loss function, further elaborated in the Discussion (Section 5).

The architecture of the current PINNs-MPF framework is modular and composed of various implementations. To better illustrate the PINNs-MPF framework, Fig. 1 presents a flowchart of the framework.

Panel (a) in Fig. 1 presents the global architecture: Similar to the conventional MPF method, full discrete training in space and time is considered. Our experiments demonstrate that this approach not only eases the comparison to our reference MPF solutions but also allows for maintaining a constant demand for resources throughout the entire simulation, in contrast to continuous training, which demands growing consumption [7]. The general control of the training is handled through a master network, referred to as the *MASTER-PINN*, while the workload is shared among multiple parallel NNs (subdomains), referred to as WORKER PINNs (c.f. paragraph 2.2). The subdividing procedure is automated. Within each subdomain, the training methodology is carried out independently, employing all distinctive techniques as introduced in the following. This includes 'pyramidal training', a structured approach designed for handling the transfer of learning along the spatial domain (c.f. paragraph 2.3). The term 'basket of PINNs' (c.f. paragraph 2.7) refers to an ensemble of networks dynamically selected for optimization to enhance efficiency. Indeed, Without an efficient strategy, the resource demand can grow exponentially, leading to impractical training times and excessive use of computational resources. Additionally, the concept of the 'wheel of optimizers' (c.f. paragraph 2.7) is introduced, denoting the interaction between the optimizers.

The training is in two directions: First in space, where the domain is decomposed into regular boxes (denoted as $N_{batches}$). The degree of this decomposition depends on the complexity of the problem. The second direction of training concerns the phase-fields. For each of these spatial–temporal blocks, referred to as a batch, a single WORKER PINN, referred to as $PINN_i$, is trained to locally predict the solution. Hereafter, the term "blocks" is employed interchangeably with "quarters", as the spatial domain is consistently divided into four blocks. Each $PINN_i$ handles a given phase in a given batch and, depending on the domain decomposition, interacts along boundaries for spatial continuity.

At the beginning of each training (time) step, the batch structure is separately built for each phase-field α, i.e., $N_{batches}$ PINNs are created and inherit the same architecture as the *MASTER-PINN*. During the training, several optimization techniques could be applied (based on the initial user selection), including pyramidal training, the wheel of optimizers, the basket of $PINN_i$s, and the concept of Quarters-based training. An upper view of the global algorithm is shown in Algorithm 1, where $N_{\text{batches min}}$, $scipy_{\text{epoch}}$ denote the minimum number of batches (coarser subdivision in case of pyramidal training) and the periodic epoch for scipy optimization respectively. In the following, we describe the details of various implementations of the PINNs-MPF framework and the related algorithm.

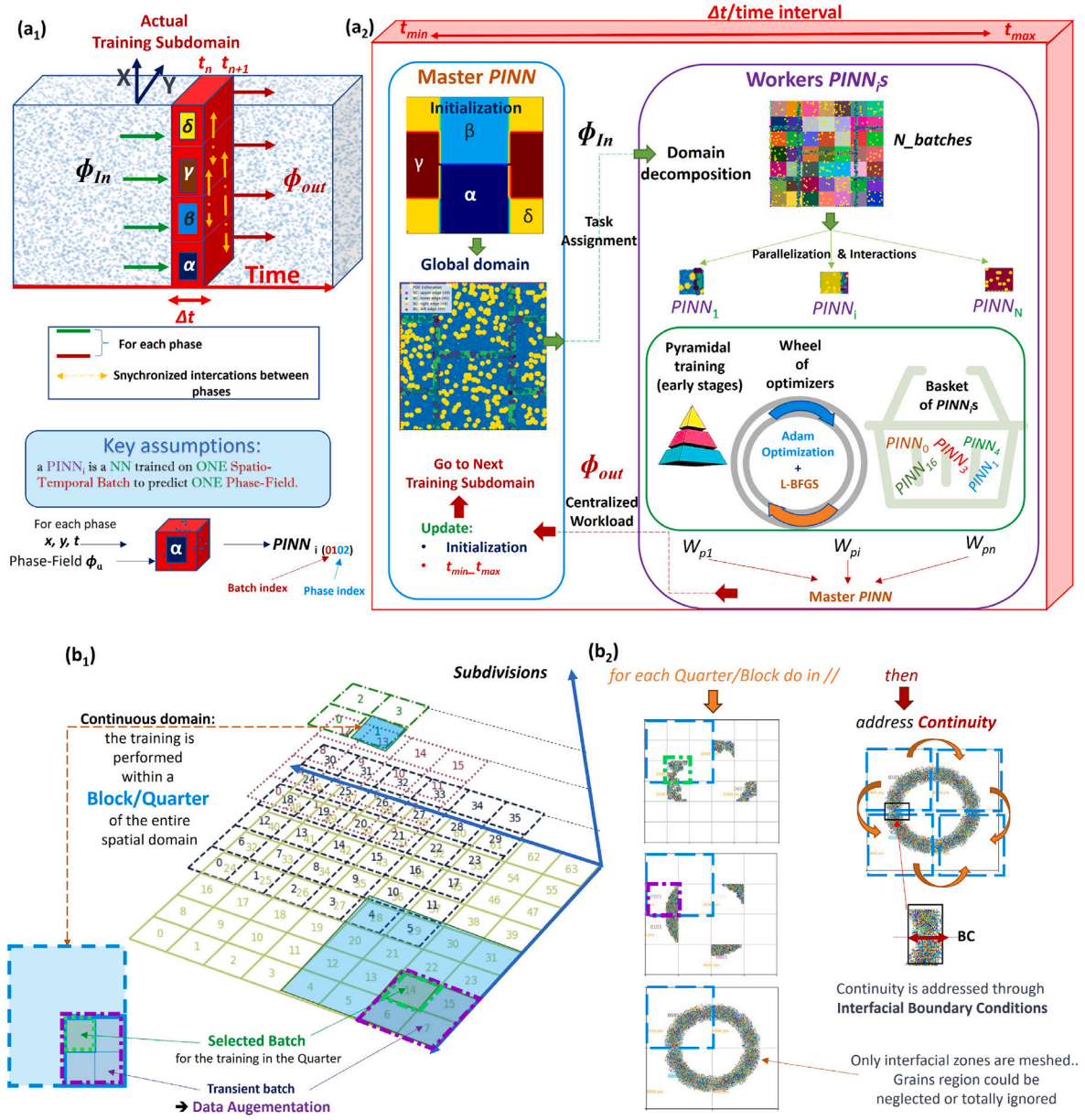


Fig. 1. The architecture of PINNs-MPF framework. Panel (a) presents the global architecture of the PINNs-MPF framework: A general description of the discrete training subdomain is provided in (a_1), while a detailed illustration of the training process within each time interval is shown in (a_2). Panel (b) presents details on pyramidal training and continuity across domains: A comprehensive description of the pyramidal transfer of learning from one subdivision level to another is depicted in (b_1), while an illustrative example of this hierarchical approach, followed by addressing continuity between domains, is provided in (b_2). The pyramidal training enables continuity and multiple-to-multiple transfer of learning. The continuity of the prediction is ensured by the propagation of boundary predictions. The set of techniques employed in this model is accessible within the code. The user has the flexibility to reserve the entire or a specific subset of the options, based on the simulation requirements and complexities.

2.1. Discrete resolution in time

We consider the MPF equations as solving a multi-variable time-series problem through the NNs. This has also been discussed in previous studies [66–69]. The computational domain in the panel (a_1) in Fig. 1 depicts this idea. Here the training process occurs in spatial domains/subdomains at fixed time intervals, analogous to time steps in conventional modeling methods, but with relatively larger intervals to harness the robust linearization capabilities of deep learning (DL). In order to proceed to a subsequent time domain, a global loss threshold is predefined to be achieved. This discrete resolution ensures a quasi-constant resource consumption, hence preventing any accumulating computational load along the time axis. To draw an analogy between discrete resolution in MPF modeling, the notation Δt (also denoted Δt^* for dimensionless resolution) is adopted here for both PINNs and MPF resolutions; in the context of PINNs, this notation refers to the length of the time interval. In the context of the PINN literature, this approach categorizes our framework within the class of time-marching or temporal PINNs, as discussed in the causality literature [34,70,71].

2.2. Extended domain decomposition

This technique extends the domain decomposition approach introduced in previous PINN studies [30,33,72] by adding a third dimension of decomposition and parallelization along the phases direction. This extension aims to handle the increased complexity of the target problem more effectively. A centralized network ensures synchronization among the multi-networks, facilitating a coordinated learning process across the decomposed spatial, temporal and phase domains. Each

Input1: Initialization
Input2: USER inputs

Initialize the Master PINN ← User inputs
Preapre Global training data ← Initialization + USER inputs
//
for *epochs in range* N_{epochs} *(total number of epochs)* **do**
  set t_{min} and t_{max}
  // → `training subdomain bounds`
  while $N_{batches} \geq N_{batches\ min}$ **do**
    initialize the multi-networks/pinns.
    decompose the subdomain.
    → Horizontal subdivision (initialization of $PINN_i$s handling first phase).
    + Vertical extrusion (initialization of $PINN_i$s handling other phases).
    affect IC training data to each $PINN_i$).
    → dynamically generate BC and collocation points for each $PINN_i$.
    divide the subdomain into different training Blocks/Quarters.
    assign each $PINN_i$ to a parent Block.
    for *each Block* **do**
      for *each* $PINN_i$ *in Block* **do**
        // `Remind: each` $PINN_i$ `handles one different phase`
        if *first phase* **then**
          select a candidate $PINN_i$ ← highest ratio of interfacial points.
          → put "selected $PINN_i$" in the Basket.
          store information about "selected $PINN_i$" (dictionary). → to use if pyramidal training
        else
          for *each phase in current Block* **do**
            get ← the NN with the same batch index as the "selected $PINN_i$".
      // ⇒ `Basket of` $PINN_i$`s Ready`

    Multi-processing → start Parallel Training → parallel optimization using Adam.
    if *epoch % scipy*$_{epoch}$ **then**
      if *pinn.loss* ≥ *Threshold for pinn in pinns* **then**
        apply L-BFGS-B optimization.
      resample collocation points.
    if $PINN_i.loss \prec$ *Threshold for* $PINN_i$ *in* $PINN_i$*s* **then**
      if $N_{batches} \succ N_{batches\ min}$ **then**
        reduce $N_{batches}$ ← Eq. (8)
        // → `Pyramidal training.`
  increase t_{min} and t_{max}
  // → `go to the next training subdomain`

Algorithm 1: General training scheme for the implemented PINNs-MPF.

phase, managed by a corresponding Neural Network, requires concurrent access to the interaction terms (I_γ) of other phases (Eq. (15)), which collectively form dual or higher-order junctions. These interaction terms demand a synchronization method (Algorithm 2). Consequently, parallel implementation is crucial for the precise calculation of interaction terms, as per the NN definitions. A serial approach cannot adequately manage these interactions, rendering parallelism indispensable rather than merely an improvement to be quantitatively compared against serial execution or existing methods, if available.

One challenging aspect of a PINNs-MPF is the heavy computational costs associated with the size of the simulation domain and the number of phase-fields. Here the concept of multi-networking is essential to distribute this computational load efficiently. This involves the parallelization of multiple $PINN_i$s and their associated subdomains obtained through a structured domain decomposition. In this context, a *MASTER-PINN* is designed to distribute tasks, training data, and collate the learned outcomes from the *WORKER-*$PINN_i$s/$PINN_i$s. It is to note that while PINNs primarily rely on minimizing a physics-based loss function and do not depend on datasets in the conventional sense, the framework dynamically generates and samples training batches, including collocation points, initial conditions (IC), and boundary conditions (BC). These elements collectively form what is referred to in the manuscript as "training data". Each $PINN_i$ operates independently and concurrently, utilizing a dedicated thread in the computer. It functions indeed as an autonomous entity responsible for processing a specific batch within the spatial domain and managing a given phase-field within the MPF domain. All $PINN_i$s are assumed to have the same architecture (number of layers and neurons per layer, learning rate, etc.). This facilitates complete parallelization of the training process and ensures spatial continuity throughout the training.

As depicted in the flowchart in Fig. 1, the *MASTER-PINN* does not undergo any training but performs several central tasks, including (i) preparing the training data set, (ii) initializing and assigning each WORKER $PINN_i$ for training, (iii) initiating and controlling the training process, (iv) facilitating communication among the WORKER PINNs, (v) managing the temporal transition of the training, and (vi) transferring the (ideally) successful learning from the actual training to another simulation. From a technical point of view, it is noteworthy that the current multi-networking approach enables the transfer of learning from multiple networks to a single network. This capability is available within the code through the pyramidal training option, wherein the subdivision could be progressively transformed from fine to coarse, ideally converging to a single network (c.f. paragraph 2.3).

The communication between $PINN_i$s could be classified into two categories: a spatial 'horizontal' communication which is required to

ensure spatial continuity across the boundary between their subdomains (c.f paragraph 2.4) and a phase-field 'vertical' communication that allows the coupled co-evolution of the phase-fields in the context of the MPF model (c.f paragraph 2.6). To facilitate the training of the *WORKER-PINN*$_i$s and their efficient management by the *MASTER-PINN*, the concept of the trinity batch-*PINN*$_i$-phase is crucial. Each *PINN*$_i$ is assigned a unique identifier by associating it with a specific batch number and phase-field number, as illustrated in Fig. 1a. Consequently, the local neural network incorporates horizontally stacked spatial–temporal coordinates (x, y, t), the output (prediction) of which is a single array representing the fraction of the corresponding phase, ranging from 0 to 1. By simplifying the multi-phase problem into discrete single-phase problems in this manner, the computational approach is streamlined. This necessitates efficient interactions between the phases, which is treated through the 'vertical' communication. Finally, it is preferred to spatially decompose the simulation into $N_{batches}$ equal batches.

2.3. Pyramidal training and block training

Merging the hyper-parameters of various *PINN*$_i$s (WORKERS) at once and with equal significance can result in overwriting certain features in the domain. For this purpose, we have thought of a gradual pyramidal training and merging. The concept of pyramidal training draws inspiration from the data augmentation principle in ML [73,74]. It is based on the assumption that adding more data to a pre-trained model serves as a warm starting point, enabling the NN to better capture crucial features before further data processing. We propose to implement a pyramidal training paradigm in conjunction with another concept of 'training on blocks'. These two complementary concepts are showcased in Fig. 1(b) and described as follows: Once the spatial domain is subdivided into multiple batches, each one can be further divided into distinct domains, giving a pyramidal structure. Each of these finer domains in the pyramidal structure is called a block or a quarter. The procedure of subdivision can continue as depicted in 1(b). After the pyramidal subdivision is completed, the training process initiates at the finest subdivision level (i.e., the base of the pyramid). However, instead of training all Neural Networks (NNs) within all batches, within each block/quarter, *PINN*$_i$ with the highest number of interfacial points is selected for training; this selection concerns the first phase and the selected *PINN*$_i$s serve as a reference for other phases. Similarly, for the other phases (in the vertical direction), training is assigned to the *PINN*$_i$ within the same batch as the reference *PINN*$_i$ within each block/quarter. This selective handling reduces the computational costs and makes the training focused on hot spots in the simulation domain. Once target losses are reached for the selected *PINN*$_i$s, the training stops at the finer level and transitions to a coarser level using the same algorithm described above. The training on the coarser level begins with a warm-start approach, where each selected *PINN*$_i$ receives learning from the previously selected *PINN*$_i$ at the finer level, within the same quarter and sharing a spatial intersection area, thereby achieving a data augmentation-like training. The transition from one level of the pyramid to another is set by a squared relationship to ensure a pyramid-like subdivision:

$$N_{\text{batches coarser}} = \left(\sqrt{N_{\text{batches}}} - 2\right)^2 \tag{8}$$

where $N_{\text{batches coarser}}$ is the number of batches in the next upper level until ($N_{batches} = N_{\text{batches min}}$) is reached, that means the continuation of training the *PINN*$_i$s on the whole spatial domain. This gradual resolution has two main advantages: On the one hand, it allows a fast identification of the optimal set of hyper-parameters, and therefore key features, without the need for an adaptive weighting strategy. On the other hand, it guarantees the training of the same numbers of NN independently of the number of subdivisions ($N_{batches}$). For that, it is worth mentioning that such a strategy is highly recommended to be activated for the early stages of the training and then set off to speed up the training.

2.4. Adaptive mesh-free optimization

The Adaptive mesh-free optimization (AMFO) is here introduced as a PINN implementation extending the adaptive mesh refinement (AMR) concept, from the phase-field literature, to dynamically adjust collocation point distributions. AMR variants have indeed proven valuable for phase-field and MPF models, allowing efficient capture of evolving interfaces and steep gradients. By incorporating adaptive collocation point sampling, this approach leverages the mesh-free nature of PINNs while retaining AMR benefits like improved accuracy and robustness [75–79]. The ease of integrating adaptive sampling with PINNs makes this approach attractive for tackling challenges posed by phase-field models, such as complex geometries and localized high-gradient regions. The proposed technique employs adaptive collocation point sampling to concentrate points around interfacial regions, a denoising loss function to reduce noise in non-interfacial areas, and dynamic resampling to continuously adapt the overall collocation point distribution.

2.4.1. Adaptive re-meshing

In the presence of diffuse interfaces, continuity among the phase-fields and across the spatial domains is required. Inspired by the storage concept proposed for the OpenPhase [64], we adopt a mesh structure that mainly focuses on the interfacial regions, neglecting interior grain zones where phase-fields are equal to 0 or 1. This strategy not only reduces the computational costs but also reduces the tendency for an error when coping for the continuity across the domains/batches. This is further promoted by the high capacity of fitting and extrapolation of ML which we elaborate on later in the Discussion (Section 5). To this end, some key hyper-parameters are user-predefined as follows. First, the minimum and maximum number of IC points per batch $N_{\text{ini min}}$ and $N_{\text{ini max}}$ are defined. Once the ratios of 0 and 1 phase-values per batch are given, the number of IC points per batch $N_{\text{ini per batch}}$ is set, following:

$$N_{\text{ini per batch}} = \min\left(\text{int}\left(\frac{N_{\text{ini max}} - N_{\text{ini min}}}{1 + e^{-\xi}} + N_{\text{ini min}}\right), N_{\text{ini max}}\right), \tag{9}$$

in which ξ donates the percentage of interfacial points per batch. The sigmoid-like function in Eq. (9) allows gradual densification of the mesh depending on the importance of the zone, so that the diffuse interfacial area and triple junctions in a multi-phase configuration should be allocated by the higher number of IC points. Using Eq. (9), the collocation points are generated dynamically around the IC points (typically within a circle of radius $\eta/2$) through a predefined ratio to get $N_f = C_f * N_{\text{ini}}$, where N_f is the number of collocation points per batch and C_f is a user parameter that could be adjusted depending on the nature of the problem. Experimenting with our benchmark studies below, we varied this coefficient between 20 to 40. Another potential improvement, to reduce the hyper-parameters in Eq. (9) and to increase the significance to triple junctions, is to define the density of interfacial points dynamically. This would involve computing the percentage of interfacial points per batch $N_{\text{ini per batch}}$ dynamically, thereby determining the number of interfacial points per batch as follows:

$$N_{\text{ini per batch}} = \left[\frac{|x_{\min} - x_{\max}| \cdot |y_{\min} - y_{\max}|}{\eta^2}\right] \cdot \xi \cdot \chi \tag{10}$$

where $x_{\min}, x_{\max}, y_{\min}, y_{\max}$ represent the limits of each *PINN*$_i$. Recall that ξ corresponds to the associated percentage of interfacial points, emphasizing that η denotes the interfacial width, and these parameters are already available within the code. Additionally, χ is an introduced hyper-parameter (user input) representing the local density of the mesh, indicating how much the mesh should be densified based on the number of interfacial points inside. With this improvement, the geometric hyper-parameters are reduced to two variables: the number

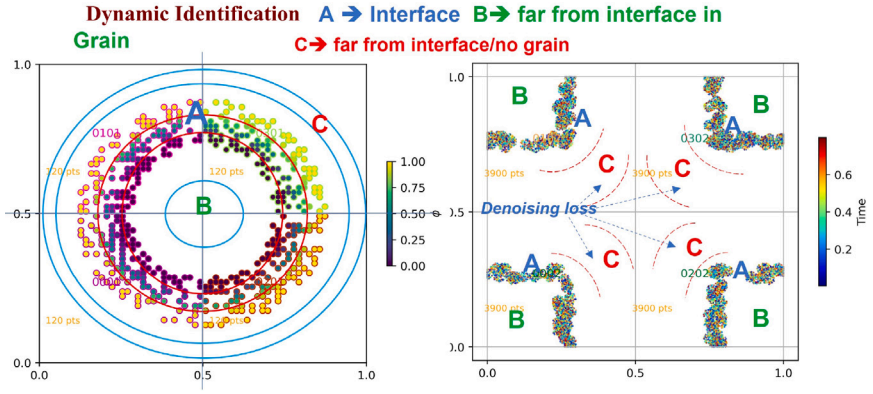


Fig. 2. Illustration of the denoising loss which is computed in the identified areas B and C, far from the interface .

of subdivisions ($N_{batches}$) and the local density (χ), considering that the C_f coefficient could be set at 20 for all numerical experiments. This simplification allows focusing on the architecture of the neural networks. The $PINN_i$s sharing the same batch index while handling different phases should use common collocation data (common batches). This introduces an additional stage of optimization. Further details on this can be found in the Supplementary Material (SM), section C.

2.4.2. Denoising loss

Focusing on the diffuse interfacial zones gives an efficient and precise approximation of the MPF solution, but still does not exclude random prediction away from the interfaces. To address this issue, a denoising loss, functioning as a corrector, is introduced. A batch is typically made of three distinct regions: A, B, and C, where region A corresponds to the interface, region B is the grain-containing area, and region C encompasses areas far from the interface (no-grain), see Fig. 2. By dynamically identifying areas of interest, as depicted in Fig. 2, the additional loss term is applied within unlabeled regions that are free of collocation points, such that, each $PINN_i$ operates to minimize the prediction of phase-field values in regions identified as 'no grain' while maximizing predictions in the grain-containing regions. Eq. (11) describes this loss, where the Mean Squared Error (MSE) is calculated based on the prediction ϕ_{pred} and is conditioned on whether the region is identified as 'no grain' or 'grain,' denoted by $\text{flag}_{\text{no grain}}$ and $\text{flag}_{\text{grain}}$, respectively:

$$\text{loss}_{\text{denoising}} = \begin{cases} \text{MSE}\ (\phi_{\text{pred}}, 0), & \text{where no grain } (flag_{\ no\ grain} = 0) \\ +\ \text{MSE}\ (\phi_{\text{pred}}, 1), & \text{where grain } (flag_{\ grain} = 1) \end{cases} \tag{11}$$

2.4.3. Dynamic resampling

Resampling, applied after each Scipy optimization, serves as a dynamic process that benefits both converged and non-converged $PINN_i$s. The collocation points undergo automatic resampling. This approach offers a dual purpose: converged $PINN_i$s are subjected to testing with new but relatively similar populations. This testing mechanism safeguards against overfitting and ensures the robustness of the predictions as confirmed by previous PINN studies [10,80–83]. Simultaneously, the non-converged $PINN_i$s are exposed to new, unexplored samples to have a better chance to enrich their training and therefore a better convergence.

2.5. Propagation of BC across sub-domains

As shown in Fig. 1(b), the propagation of BC along the neighbors in our decomposed space is required to ensure continuity and therefore a reliable prediction of the MPF solution. To this end, a customized BC loss is introduced that concerns the spatial internal boundaries across subdomains/batches/$PINN_i$s handling a given phase-field. Here, each $PINN_i$ optimizes its weights by identifying its neighbors while considering the boundary predictions of its neighbors that are used to minimize the additional BC loss. The BC loss function is set as the sum of differences between the prediction of the current $PINN_i$ and those of the neighbors, depicted also in Fig. 1b. A detail that enhances the handling of the boundary condition is to consider the $PINN_{00xx}$ (in batch 0 for each phase) as an absolute reference after addressing its PDE and IC losses. Subsequently, for a given $PINN_i$, the west and inner neighbors are regarded as temporal references. This leads to a systematic propagation of the boundary conditions, ensuring that all $PINN_i$ instances update their weights based on the $PINN_{00xx}$. These interactions create mutual dependencies among neighboring domains/batches/$PINN_i$s, fostering spatial–temporal continuity during the training process.

2.6. Handling multi-phases (vertical optimization)

The full discrete construction of the current framework in space, time and phase-fields allows the instant communication between $PINN_i$s, through the *MASTER PINN*, that is required to perform the multi-phase studies. This is achieved in two steps: In the first step, one is entailed to identify the possible interaction(s) between any two phase-fields. This must be done for every $PINN_i$. Obviously, with every interaction found, an associated $PINN_i$ for that phase-field is identified as well. A fundamental assumption here is that two given phase-fields interact when their diffuse interfaces intersect. This results in obtaining a batch-phase space. In 2D, a maximum of three phase-fields intersect, forming a triple junction. An example of this idea is illustrated in Fig. 3 where we find that out of the 48 possible batch-phase combinations, 8 exhibited no interactions. This approach has a significant impact when handling a large number of phase-fields. Hereafter, the term 'grain' is also used interchangeably with 'phase' once reflects the same entity.

The *MASTER-WORKER* $PINN_i$ structure enables a computational- and memory-efficient calculation of the Laplacian term in Eq. (1). Each $PINN_i$ responsible for a specific phase-field communicates its computed Laplacian term instantly to the neighboring $PINN_i$s working on the same batch but handling different phase-fields. This is essential for avoiding the computation of multiple Laplacian terms by each $PINN_i$ and enhancing the computational efficiency. The numerical implementation of the MPF model (Eq. (1)) is simplified into the parallel computation of the interaction terms for each phase within each batch. Subsequently, a robust communication protocol ensures synchronization of the computation for the correct equation terms, as detailed in Algorithm 2. To ensure synchronization, a given $PINN_i$ needs to wait to receive all the required information from the $PINN_i$s in interactions. As a reminder (c.f. paragraph 2.4), an optimization detail involves the use of a shared collocation dataset for all $PINN_i$s handling different phases within the same batch. For simplification, the collocation dataset corresponding to the first phase in the initialization (phase α in algorithms 1 and 2) is consistently selected as a reference.

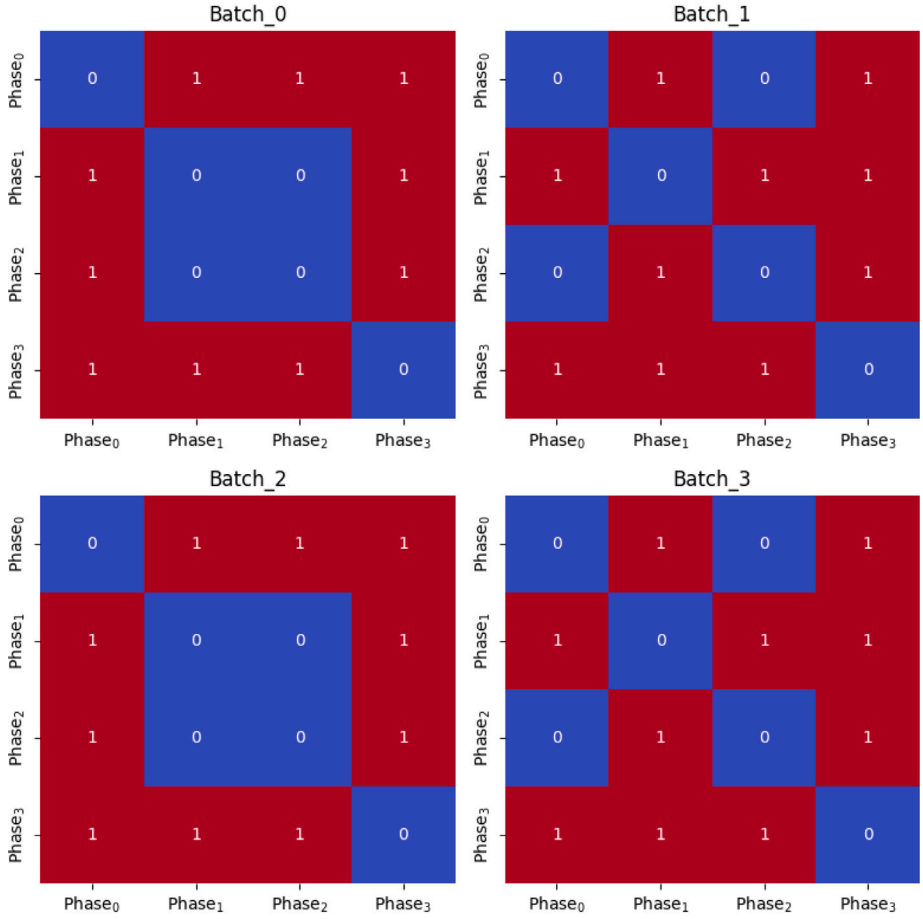


Fig. 3. Illustration of the interaction between different phases in different batches for a 2D triple junction case: each phase, in each batch interacts with up to three phases.

2.7. The wheel of optimizers

The framework effectively combines Adam's stochastic optimization with Scipy's L-BFGS, fostering a collaborative approach. Adam operates at each epoch, contributing its stochastic optimization capabilities, while Scipy, notably the L-BFGS optimizer, is periodically selected. During these selections, L-BFGS efficiently leverages information from the Hessian matrix to optimize weights. When a given $PINN_i$ successfully converges, reaching a loss value below the predetermined threshold, it temporarily steps back, entering a wait state. Meanwhile, other $PINN_i$s, selected from a shared 'basket of $PINN_i$s,' proceed with their L-BFGS optimization. This metaphorical basket illustrates the practice of choosing specific PINNs for weight optimization and then reintegrating them, symbolizing a cyclic process; hence, the terminology 'wheel of optimizers'. The purpose of this cyclic process is to ensure that optimized learning results in synchronized motion between batches, leading to accurate predictions at boundaries, and effectively captures the dynamics across phases, including terms from PDE.

2.8. Transfer of learning

The highly symmetric and parallel construction of the current implementation enables the transfer of learning in multiple directions. Within the same simulation, we transfer the learning spatially through the pyramidal training (between the levels of pyramids) and across boundary conditions. The transfer also occurs temporally, from one spatial domain to its image in the next time step. These allow us to create checkpoints to continue/restart a simulation without losing the accumulated learning. A particular application of such transfer is to optimize the parallel performance of the worker $PINN_i$s. When a worker $PINN_i$ fails to converge after a certain number of Scipy optimizations, it is then warm-started by receiving the learning from the nearest neighbor already converged. This is to enhance its convergence rate and overall performance. Such a situation could arise indeed due to various factors, including insufficient data or training samples, numerical instabilities, singularities or discontinuities in the data and inadequate regularization. Additionally, the framework allows real-time processing, facilitating easy debugging and dynamic adaptability.

2.9. Phase correction

As a requirement to handle MPF problems, a dynamic correction mechanism for phase predictions is hereafter introduced. This algorithm conserves the summation of the phase fields of different phases within interfacial regions. PINNs often fail to predict correct solutions, necessitating correction algorithms to satisfy conservation laws [47]. Therefore, incorporating conservative correction algorithms and adequate optimization methods is essential for accurate predictions [44].

3. Multi-phase-field method

The free energy functional for a polycrystalline domain is written as

$$F = \int_{\Omega} f^{GB} + f^{Ext} \tag{12}$$

with f^{GB} is the grain boundary free energy density [$J.m^{-3}$] and the f^{Ext} is to emphasize the presence of other energetic contributions (e.g. chemical and elastic energy densities) which are not specifically

Input1: PDE equation ← Eq. (1)
Input2: Interaction infos (dictionary)
if *phase 1* **then**
  get ← self $batch_{Xf}$
else
  identify the other $PINN_i$s in interaction with self.
  get $batch_{Xf}$ of the $PINN_i$ handling the phase 1.
compute $\dot{\phi}_\alpha$
compute self interaction term (I_α) ← Eq. (15)
for $PINN_{i\ beta}$ *in* $PINN_i$*s in interaction* **do**
  // $\alpha \neq \beta$
  while I_β *is not updated* **do**
    wait
  for $PINN_{i\ beta}$ *in* $PINN_i$*s in interaction* **do**
    // $\gamma \neq \alpha, \beta$
    while I_γ *is not updated* **do**
      wait
  All interaction terms are computed
  compute the right side of Eq. (1)
  compute the PDE loss for the actual batch ← Eq. (4)
  correct the phase predictions ; $\phi_\alpha \leftarrow \frac{\phi_\alpha}{\sum_{i=1}^{N}\phi_i}$
  compute the sum constraint loss for the entire grid at $t = t_{max}$ ← Eq. (7)
  return the total loss

Algorithm 2: The vertical optimization along phases to optimize the computing of the PDE loss for a given PINN$_i$s (prediction a given phase (α) in a given batch). Here "self" denotes the considered PINN$_i$ while $batch_{Xf}$ represents the associated dataset of collocation points. The dynamic correction of the phase summations is done within this block.

considered here. The grain boundary free energy density that is given as [49]

$$f^{GB} = \sum_{\alpha,\beta=1..N,\alpha>\beta} \frac{4\sigma_{\alpha\beta}}{\eta}\left\{-\frac{\eta^2}{\pi^2}\nabla\phi_\alpha\cdot\nabla\phi_\beta + \phi_\alpha\phi_\beta\right\} \tag{13}$$

Here ϕ_α is the non-conservative phase-field variable corresponding to the phase (grain) α (idem for β), $\sigma_{\alpha\beta}$ [$J.m^{-2}$] is the grain boundary energy and η is the width of the grain boundary interface [m]. The product $\nabla\phi_\alpha\cdot\nabla\phi_\beta$ accounts for the diffuse nature of the grain boundary and its sum asymptotically grows when going from a simple grain boundary to three and higher order junctions.

The evolution of the different grains is determined by the following system of equations:

$$\dot{\phi}_\alpha = -\sum_{\beta=1..N}\frac{\pi^2}{4\eta N}\mu_{\alpha\beta}\left(\frac{\delta F}{\delta\phi_\alpha} - \frac{\delta F}{\delta\phi_\beta}\right) \tag{14}$$

with $\mu_{\alpha\beta}$ defined as pairwise grain boundary mobilities. Inserting the free energy functional F, one obtains

$$\dot{\phi}_\alpha = \sum_{\beta=1..N}\frac{\mu_{\alpha\beta}}{N}\left[\left\{\sigma_{\alpha\beta}(I_\alpha - I_\beta) + \sum_{\gamma=1..N,\gamma\neq\alpha,\gamma\neq\beta}(\sigma_{\beta\gamma}-\sigma_{\alpha\gamma})I_\gamma\right\} + \frac{\pi^2}{4\eta}\Delta g_{\alpha\beta}\right]$$

Here the additional term $\Delta g_{\alpha\beta}$ accounts for the non-interfacial contributions to the free energy functional, referred to as a driving force throughout this paper. Further, we have

$$I_\alpha = \nabla^2\phi_\alpha + \frac{\pi^2}{\eta^2}\phi_\alpha \tag{15}$$

The phase-field variables are assumed to respect the sum constraint:

$$\sum_{\alpha=1..N}\phi_\alpha = 1 \tag{16}$$

which is included in the training as an additional but critical loss term. Indeed, the phase prediction should be dynamically divided by the sum of the order parameters of all phases.

In our simulations, we have considered isotropic interfacial mobility (μ) and energy (σ) also a constant driving force. This results to the simplified form of $\dot{\phi}$, presented in Eq. (1). The latter consideration simplifies $\Delta g_{\alpha\beta} = \Delta g.\nabla\phi$ where Δg is a constant value. In case of a single grain boundary (two phase-fields system):

$$\dot{\phi} = \mu\left[\sigma\left(\nabla^2\phi + \frac{\pi^2}{2\eta^2}(2\phi-1)\right) + \frac{dh}{d\phi}\Delta g\right] \tag{17}$$

with

$$\frac{dh}{d\phi} = \frac{\pi\sqrt{\phi(1-\phi)}}{\eta}. \tag{18}$$

and the kinetic coefficient of the interface L: $\mu = \frac{8\eta L}{\pi^2}$

For a triple junction, the Young's law becomes

$$\frac{\sin\theta_{12}}{\sigma_{12}} = \frac{\sin\theta_{13}}{\sigma_{13}} = \frac{\sin\theta_{23}}{\sigma_{23}}, \tag{19}$$

3.1. Dimensionless form of the PDEs

The convergence and training speeds of the model are strongly linked to the chosen physical parameters. The use of dimensionless representations for the PDEs is recommended when dealing with slow training. The non-dimensional time, spatial coordinate, temporal derivative, and dimensionless gradient as proposed as follows.

$$\tau^* = \frac{t}{T};\quad x^* = \frac{x}{\eta};\quad \dot{\phi}^* = \frac{\dot{\phi}}{T};\quad \nabla^* = \frac{1}{\eta}\frac{\partial}{\partial x}\mathbf{i} \tag{20}$$

where the time characteristic T is identified:

$$T = \frac{\eta^2}{\mu\sigma} \tag{21}$$

while η represents the characteristic length. Thus, the MPF model, as represented by Eq. (1), can be simplified for numerical purposes:

$$\dot{\phi}_\alpha^* = \sum_{\beta=1..N\neq\alpha}\frac{1}{N}\left[\left\{(I_\alpha{}^* - I_\beta{}^*) + \frac{\pi^2}{4}\Delta g_{\alpha\beta}{}^*\right\}\right] \tag{22}$$

with:

$$I_\alpha{}^* = \nabla^{*2}\phi_\alpha + \pi^2\phi_\alpha \tag{23}$$

Similarly, the dual-phase case (Eq. (17)) could be simplified to:

$$\dot{\phi}^* = \left[\left(\nabla^{*2}\phi + \pi^2(2\phi-1)\right)\right] + \frac{dh}{d\phi}\Delta g^* \tag{24}$$

4. Simulation results

Analogous to the foundational developments of OpenPhase [64], the reliability and performance of a PINNs-MPF for studying the interface dynamics in polycrystalline materials need to be systematically benchmarked to capture three key features of MPF approach: First, the curvature-driven motion of an interface, second, the corrected motion of the same interface under any additional driving force and third, the force balance at the junctions of interfaces [64]. The first two requirements in this set can be generally written as:

$$v_n = \frac{\dot{\phi}}{\nabla\phi} = \mu\left(\sigma\kappa + \Delta g\right), \tag{25}$$

with v_n the local interface velocity normal to its plane, κ the local curvature of the interface, and Δg a constant driving force, where with consider isotropic interface energy and mobility and, $\Delta G = \nabla\phi\Delta g$ in Eq. (1). The third requirement concerning interface junctions necessitates the MPF to fulfill Young's law, which relates the angles between interfaces at the junction to the interfacial energy. For a triple junction and isotropic interface energy assumed, the three interfaces at equilibrium meet at a 120° angle. To test the PINNs-MPF framework, certain simulation scenarios related to the above requirements are studied as listed in Table 1, along with related physical parameters and applied hyper-parameters in the PINNs implementation in Table 2. We

Table 1
Numerical and physical parameters of the testing benchmarks. The corresponding units of the interface width, mobility, and interfacial energies are respectively m, $m^4J^{-1}s^{-1}$, and Jm^{-2}.

	Spatio-temporal parameters (dimensionless)					Physical parameters			
Benchmark	Nx	dx	Ny	dy	Nt	Interface width	Mobility	Interfacial energy	Phases
Traveling wave interface	100	0.01	100	0.01	1000	10dx	1e−4	1	1
Grain shrinkage under driving force	64	..	64	..	100	7dx	1e−4	1	1
Curvature-driven grain shrinkage	64	..	64	..	150	4-9dx	1e−4	1	1
Triple juncion	64	..	64	..	up to 200	6dx	1e−4	1	4

Table 2
Hyper-parameters of the implemented PINN for the different testing benchmarks. LR: Learning Rate, Per.: Periodicity/Epochs.

	NN Architecture			Optimizers			
Benchmark	NNs	HL/NN	Nodes / layer	Adam		L-BFGS	
				LR	Activation function	max Iter	Per.
Traveling wave interface	1	6	32	1e−3	tanh + sigmoid (last layer)	1000	100
Grain shrink under driving force	4	6	32–128	1e−4	idem	1000	50
Grain shrink with natural motion	4	6	128	1e−4	idem	1000	50
Triple junction (*)	16 (**)	6	128	1e−5	idem	500	50

(*) For the triple junction, the pyramidal approach was tested for the early stages of the training.
(**) When using the pyramidal training, 64 NNs are inititalizated, while 16 NNs are selected for training using the concept of the basket of PINNs.

note that for the grain shrink scenarios, the number of IC points per batch $N_{\text{ini per batch}}$ serves as an input parameter, and Eq. (9) is used for its generation (with $N_{\text{ini min}}$ and $N_{\text{ini max}}$ varied up to 90). However, for the triple junction scenario, this process is automated through Eq. (10), where χ is set to 40 in order to dynamically populate the batches during training, depending on the evolution of the solution.

To further highlight the effectiveness of multi-networking in conjunction with a combination of optimizers for training PINNs, an additional numerical experiment is proposed as follows: reproducing the benchmark of grain shrinkage under constant driving forces with varying numbers of hidden layers and neurons per layer. Specifically, a fixed number of six hidden layers with the number of neurons ranging from 64 to 512 was applied. This was achieved by employing a cyclic optimization strategy alternating between the Adam optimizer and the L-BFGS optimizer (referred to as the wheel of optimizers). Associated findings and comments are provided in the discussion section.

Additionally, we illustrate in Table 3 the progressive increase of the optimization techniques to deal with the induced complexity by the tackled scenarios. The whole PINNs-MPF framework, together with the presented benchmarks in the following, is open-sourced and verified as an oceancode capsule [84].

Furthermore, for the scenarios involving grain shrinking and triple-junctions, the MPF solutions from OpenPhase calculations are used [64]. The associated scheme is provided in the SM, section A. All numerical implementations were coded using TensorFlow and performed on standard workstations with an AMD Ryzen Threadripper PRO 5975WX 32-Cores CPU (64 threads).

4.1. Traveling interface under constant driving forces

As a key starting point, the behavior of a planar interface traveling under a given driving force is explored. In the absence of the curvature, this can be simplified to a 1D problem in which the phase-field profile propagates according to [49,64]:

$$\phi(x,t)=\begin{cases}1 & \text{for} \quad x<v_nt-\frac{\eta}{2}\\ \frac{1}{2}-\frac{1}{2}\sin\left(\frac{\pi}{\eta}\left(x-v_nt\right)\right) & \text{for} \quad v_nt-\frac{\eta}{2}\le x<v_nt+\frac{\eta}{2}\\ 0 & \text{for} \quad x\ge v_nt+\frac{\eta}{2}\end{cases} \tag{26}$$

The results of this campaign are gathered in Fig. 4. The utility of analyzing a traveling wave solution lies in its ability to accurately predict the shape and velocity of the phase-field profile. Fig. 4(a) presents the setup of the BC and IC points, as well as the placement of PDE collocation points for this problem. As described in paragraph 2.4, the meshing is predominantly focused within the interface regions, where the PINN model is trained to predict the solution. Excellent agreement is obtained between the PINNs-MPF and the theoretical solutions at various time instants, Fig. 4(b). Fig. 4(c) shows the spatial evolution of the interface resolved by the PINNs-MPF. To trace the interface motion and its velocity, a representative point with a phase-field value $\phi=0.5$ at $x,t=0,0$ is selected. Fig. 4(d) compares the theoretical and predicted velocities, revealing an accuracy approaching unity after 2500 epochs of training, with a total ≈10 min of computation time.

Table 3
Enumeration of applied optimization techniques with increasing complexity of target problems, as well as the associated number of model trainable parameters.

Benchmark	Activated techniques	Number of optimization Techniques	Number of trainable parameters
Traveling wave interface	Wheel of optimizers (Adam and LFBGS), resampling, discrete or continuous resolution (optional), transfer of learning	4	3,297
Grain shrinkage under driving force	Wheel of optimizers, discrete resolution, extended domain decomposition, Adaptive Mesh-Free Optimization (AMFO), transfer of learning	5	416,005
Curvature-driven grain shrinkage	Wheel of optimizers, discrete resolution, extended domain decomposition, AMFO, transfer of learning	5	416,005
Triple junction	Wheel of optimizers, discrete resolution, Extended domain decomposition, transfer of learning, AMFO, Basket of PINNs, Handling multi-phases, pyramidal training correction of phases predictions	9	1,414,417

Table 4
Quantitative comparison between each of the tested architectures and the theoretical solution.

Configuration	Neural networks	HL	Neurons/HL	MAE	MSE
1	4	6	32	4.72×10^{-2}	3.06×10^{-3}
2	4	6	64	3.12×10^{-2}	1.47×10^{-3}
3	4	6	128	2.82×10^{-3}	1.14×10^{-5}

For a planar interface (no curvature), the interface velocity is expected to linearly scale with the input driving force (Eq. (25)). Fig. 4(e) shows the interface velocities obtained for various driving forces, perfectly matching the theoretical expectation. Here the PINNs-MPF were first trained for a single Δg value (Fig. 4(f)) and the learning is then transferred to predict the interface velocity for various Δg values. Such transfer of learning is typically carried out within a significantly reduced number of (maximum 100) epochs of training, as the training is not always evitable [85]. From a technical perspective, this analysis tests localized meshing, the transfer of learning and full-discrete resolution. It is worth noting that here a single neural network effectively handles the 1D scenario.

One can immediately expand the driving-force-driven interface kinetics to 2D where the planar interface is replaced by a circular interface. In this scenario, the interface has a curvature, but its effect is negligible under the condition that $\Delta g \gg \sigma\kappa$ (Eq. (25)). Although the linear scaling of the interface velocity holds, this scenario imposes a challenge for the PINNs-MPF going from a cartesian to a spherical coordinate. PINNs-MPF predictions versus theoretical solution from OpenPhase are shown in Fig. 5.

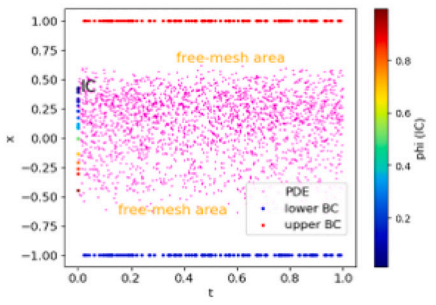
Here initial trials involving a single NN (used for the 1D case above) revealed limitations marked by training instabilities and noise (c.f. the SM for qualitative and quantitative comparison between PINN predictions and ground truth solution for a single NN). However, stable training was obtained when applying four neural networks. Indeed, subsequent refinements increased the prediction accuracy by progressively increasing the number of neurons starting from 32, resulting in an optimal configuration of 128 neurons and 6 hidden layers, as observed in Fig. 5. To quantify the differences between each PINN prediction and the theoretical solution, it is proposed in Table 4 measures of the MSE and Mean Absolute Error (MAE) for each prediction compared to the ground truth solution. The architecture with 128 neurons not only accurately matched the theoretical solution (with MAE and MSE values of 2.82×10^{-3} and 1.14×10^{-5} respectively) but also outperformed the phase-field solution obtained using an explicit scheme. Thus, it is hereafter proposed to keep this architecture for subsequent benchmarks as it allowed to deal with the double-well potential term and the non-convex potential term in Eq. (17) on a one hand, and to fix the related set of parameters of this architecture, especially for the multi-phase field scenario.

It is worth noting that changing from the 1D to 2D setup, the PINNs-MPF is well capable of capturing and maintaining the correct interface thickness throughout the simulation.

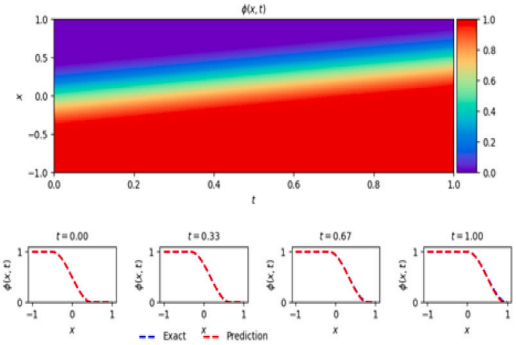
4.2. Curvature-driven interface motion

Capturing the natural curvature-driven motion of an interface is the most central capability of a phase-field model, encoded into the gradient energy term and related Laplacian in the governing free energy functional and equations of motion, respectively. When the Laplacian terms take control over the interfacial motion, i.e., $\sigma\kappa \gg \Delta g$ in Eq. (25), nonlinear curvature-driven dynamics arise: Here we study the shrinkage of a circular phase (grain) with $\kappa = 1/R$, giving $v_n = dR/dt = M\sigma/R$. This gives another level of complexity to test the PINNs-MPF framework. The number of time intervals required for the resolution of this case was 150, corresponding to $\Delta t^* = 10^{-3}$, while $\Delta t^* = 10^{-4}$ is required to resolve the same problem using the PF scheme (c.f. the SM, section A). The results are presented in Fig. 6. Here, the PINN solution fits the theoretical one. However, the phase-field linearized scheme results in a premature shrinkage of the grain. The same result is obtained even when trying with smaller time steps. This case, where the motion is fully governed by the interface and no external driving force is applied, demonstrates that PINNs can efficiently handle non-linearities in the phase-field context. Indeed, it is worth reminding that we use the tanh activation function in hidden layers to capture diverse non-linear patterns in physics-informed input data. For the output layer, the sigmoid activation ensures bounded predictions, making it suitable for interpreting outputs. The specific limits (the order parameter) remain between 0 and 1. These tailored non-linear activations allow PINN to effectively handle the grain shrinkage scenario where the motion is controlled by the interfacial energy and curvature.

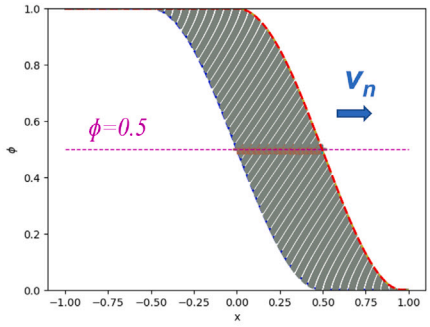
The pattern of the phase-field solution depends on the interface width λ; for a thin interface ($\eta = 4dx$), there is a slight deviation from the theoretical solution, while the results improve with increasing the width. This behavior is systematically investigated [64], showing that it arises from the numerical limitations of calculating the Laplacian and consequently the interface curvature. This is a feature of the diffuse phase-field interfaces. The PINNs-MPF predictions were studied for three interface widths of η = 4dx, 7dx and 9dx. In all configurations, a consistent alignment is evident between the PINNs-MPF and the OpenPhase solutions. This is best visible for the thinnest interface



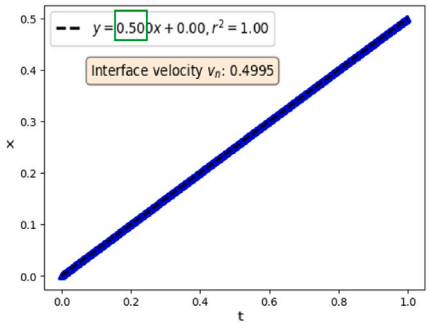
(a) Initial mesh



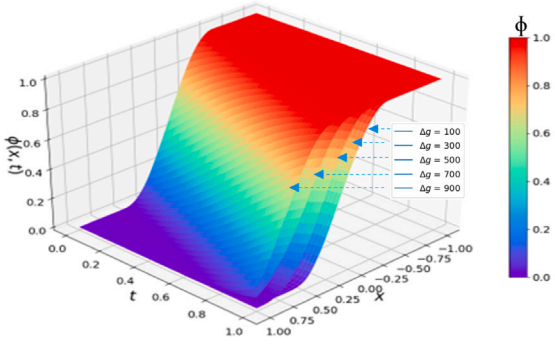
(b) Travelling wave solution



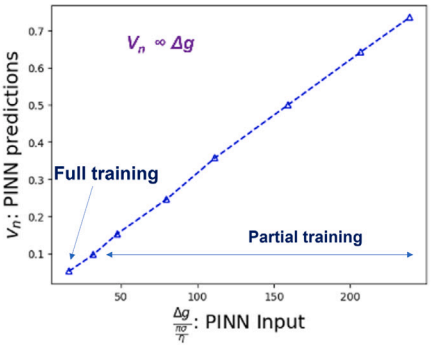
(c) Spacial motion of the interface



(d) Identification of the interfacial velocity



(e) Impact of external driving force



(f) PINN extrapolations

Fig. 4. PINN solution for the traveling wave interface problem against the theoretical solution.

shown in Fig. 6b. This set of benchmarks demonstrates that the PINNs-MPF is not only capable of handling the nonlinear interface kinetics but also has a good sense of the interface width and its impact on the solution. The latter is crucial for future developments of the model to deal with interfacial phenomena, such as interfacial elasticity and solute segregation [86,87].

4.3. Establishing equilibrium triple junctions

A major capability of MPF is to handle interfacial junctions. This is not only to fulfill Young's law but also to ensure the evolutionary path leading to a configuration with minimum energy. To test our PINNs-MPF framework, a system with four phase-fields (grains) and six triple junctions was considered. The initial mesh is visualized in Fig. 7. Using our method (c.f. the SM, section C), the IC points are densified within the interfacial region, while some random points were also selected away from the interfaces to allow PINNs to learn better. To streamline computations, BC are selectively addressed along the interface lines between different batches. It is worth highlighting that the initialization having sharp angles presents a challenge for PINNs to produce accurate solutions. This is because of the curvature-driven nature of the interface dynamics, thus, initiating away from smooth curves/shapes results in complex evolutionary scenarios.

The simulation results are visually depicted in Fig. 8. Here, each grain is shown in its initial and final state of the simulation, compared to the simulation results from OpenPhase. Fig. 8(m)–(o) show the total interfacial region in the initial and final state. The comparison between PINNs-MPF and OpenPhase reveals an excellent agreement. The microstructure evolves into an equilibrium state where the angles at the triple junctions approach 120°. One can see that the motion of the phases is well synchronized, allowing the global solution to converge to the equilibrium state. Note that the application of boundary conditions on the outer boundary of the simulation box requires the interfaces on both ends to adjust in an energy-minimizing manner.

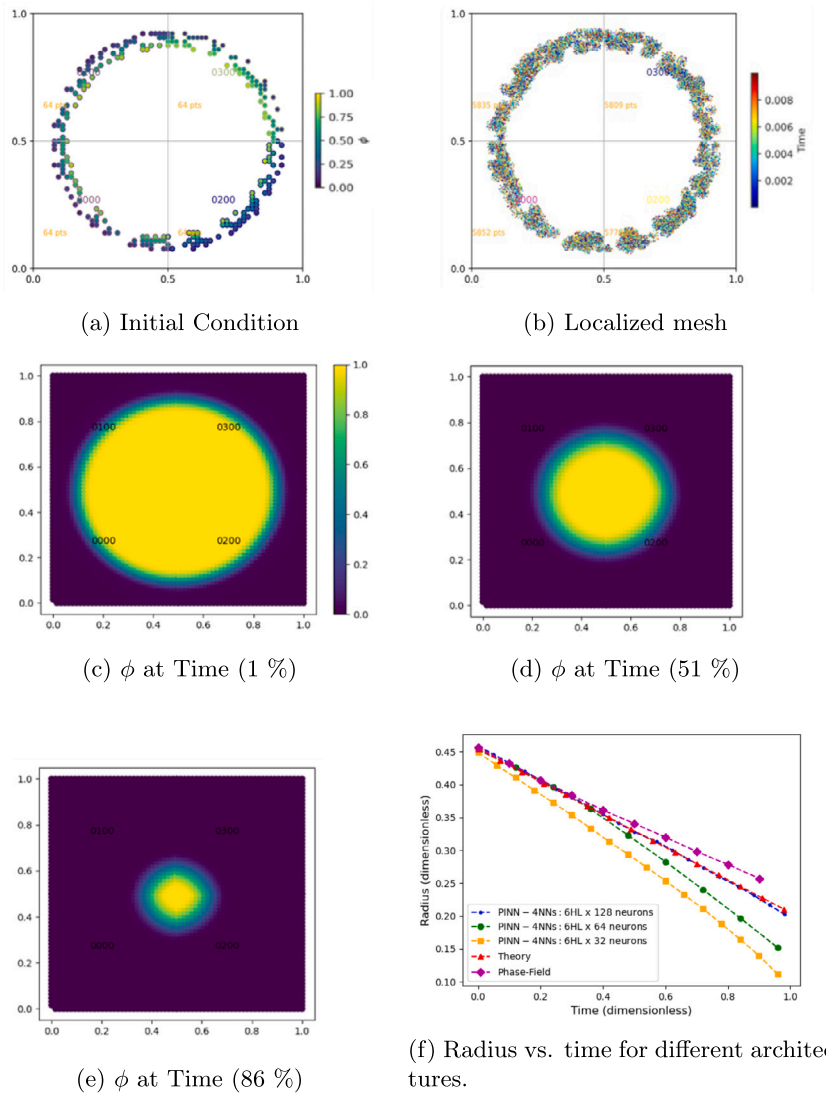


Fig. 5. Grain prediction of PINN against theoretical and PF solutions.

The difference between the PINNs-MPF solution and OpenPhase lies in the interface thickness. This difference may be justified by the gradient topology of each methodology: It is indeed noted that the gradient calculation employed in the implementation, specifically using the "gradient.tape()" module in TensorFlow, differs from the Laplacian used in the regular grid computation for MPF calculations. Additional details about the difference in the computation of the Laplacian terms are given in the SM, section A. This difference explains the minor discrepancies related to the interface width. However, it can be asserted that PINNs predictions exhibit greater fidelity compared to OpenPhase when preserving similar interface widths to the initial state. We also note that, due to the curvature-driven nature of grain growth problems, handling an initialization with sharp angles (90°) is challenging. This demonstration of the correct evolution of such a critical initialization consolidates the use of such a framework in increasingly difficult contexts. From a technical standpoint, it is worth noting that this results in relatively slow training in the beginning until addressing the sharp angles, after which the training accelerates.

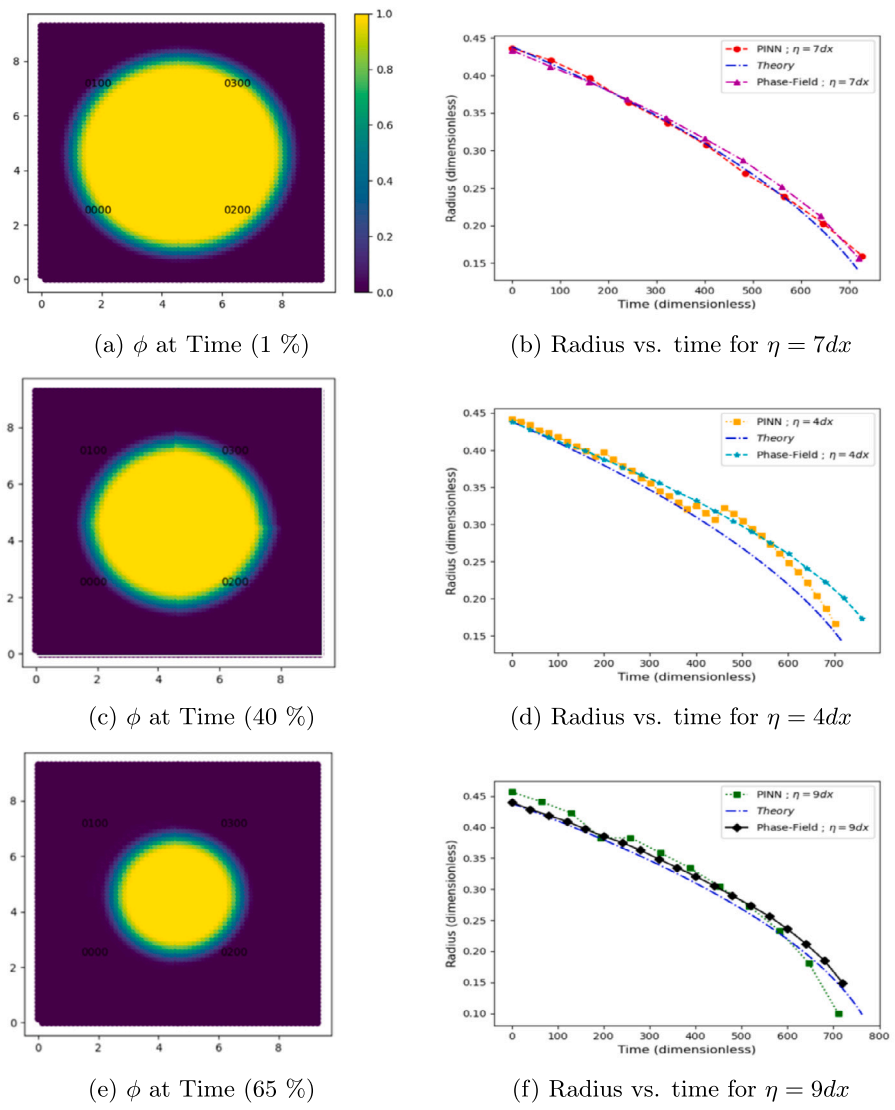
5. Discussion

Analyzing the training dynamics through the loss functions.

The training of PINNs-MPF involves various loss terms, described through Eq. (4) to (7). To understand and optimize the training process necessitates a comprehensive analysis of each loss in terms of their individual behavior and collective synchronization.

For the grain shrinking scenario (paragraph 4.2), the evolution of various mean losses for the four $PINN_i$s across epochs is shown in Fig. 9. Here, (i) each loss curve corresponds to the average of the four corresponding losses from the four neural networks (NNs) and (ii) the entire training process is technically conducted in a single execution. However, for the sake of clarity in illustrating the evolution of losses, we choose to present the simulation in two stages, such that, after convergence in the initial time interval, the acquired learning is then applied to restart the simulation.

Fig. 9a shows the evolution of the different losses for the first time interval (t_0-t_1). The threshold is set at 5×10^{-5}, marking the transition point to the next time interval. The training initially focuses on the interfacial regions, activating only the PDE and IC losses. Once the solution is computed in these regions, BC and denoising losses are subsequently activated. This aims to establish spatial continuity between batches and eliminate noise within both grain and non-grain areas, respectively. The discrete activation of BC and denoising loss has proven to be more efficient in enhancing training compared to continuous activation. It is proposed to categorize the losses into intrinsic losses, including partial PDE and IC losses, representing core elements for the NN. On the other hand, extrinsic losses, here BC and denoising, play a role in fine-tuning the solution and establishing continuity with



(a) ϕ at Time (1 %) (b) Radius vs. time for $\eta = 7dx$

(c) ϕ at Time (40 %) (d) Radius vs. time for $\eta = 4dx$

(e) ϕ at Time (65 %) (f) Radius vs. time for $\eta = 9dx$

Fig. 6. Grain radius prediction of PINN against theoretical and PF solutions (Open-Phase) for a natural motion. The plots correspond for PINN prediction for an interface width $\lambda = 7dx$.

neighboring NNs. These extrinsic losses could be subjected to optimized integration.

Fig. 9(b) shows the local minimums corresponding to this transition, where all maximum losses of $PINN_i$s (as defined in Eq. (3)) fall below this threshold, indicating the onset of training for the next time intervals. The gaps between peaks correspond to the length of each time interval. The simulation could be then divided into two distinct phases. Initially, when the grain radius is substantial, the training progresses relatively fast. Subsequently, in the second phase with a smaller grain size, the BC loss exhibits an inverse relationship with the grain radius, becoming the most influential. The seamless transfer of learning between time intervals becomes evident as the PDE loss initially tends to decrease and then stabilizes in the later stages of training. Notably, the magnitudes of losses appear reasonable, with the PDE loss being approximately ten times greater than that of IC. If the BC loss remained activated throughout the entire simulation, in the subsequent benchmark (triple junction), it was managed through discrete activation, mirroring the approach adopted during the initial time interval. We note that, in the PINN literature, the usage of the time-marching strategy to avoid causality violations in our time-dependent PDEs may have the disadvantage of leading to accumulated errors in long-duration simulations, potentially affecting prediction accuracy. However, this is applied here with additional care. First, given the relatively short time intervals required for PINN simulation (e.g., 150 intervals for curvature-driven grain shrinkage compared to 1500 in traditional solvers), cumulative error is minimized. Second, a very low loss-magnitude threshold (on the order of 10^{-4} MSE) is enforced to transition between time intervals, ensuring high accuracy at each step and limiting error propagation. Incorporating Recurrent Neural Networks (RNNs) offers a promising avenue for capturing long-term dependencies and unifying the temporal solution. This approach could enable efficient handling of long-duration simulations by maintaining memory across time intervals, left for a future study.

For the triple junction scenario (paragraph 4.3), the evolution of various losses (PDE, IC, BC, and mean loss) for the 16 NNs across epochs is illustrated in Fig. 10. In this context, the threshold is now set at 6×10^{-4}. Similar to the grain shrinking scenario, the first time interval is addressed, and then the cumulative learning is used to restart the simulation. For this purpose, two approaches are compared, with and without a pyramidal approach (respectively in Figs. 10 a and b). It is noted that the only difference between the two simulations is the number of batches for the fine subdivision $N_{\text{batches min}}$ (c.f. Algorithm

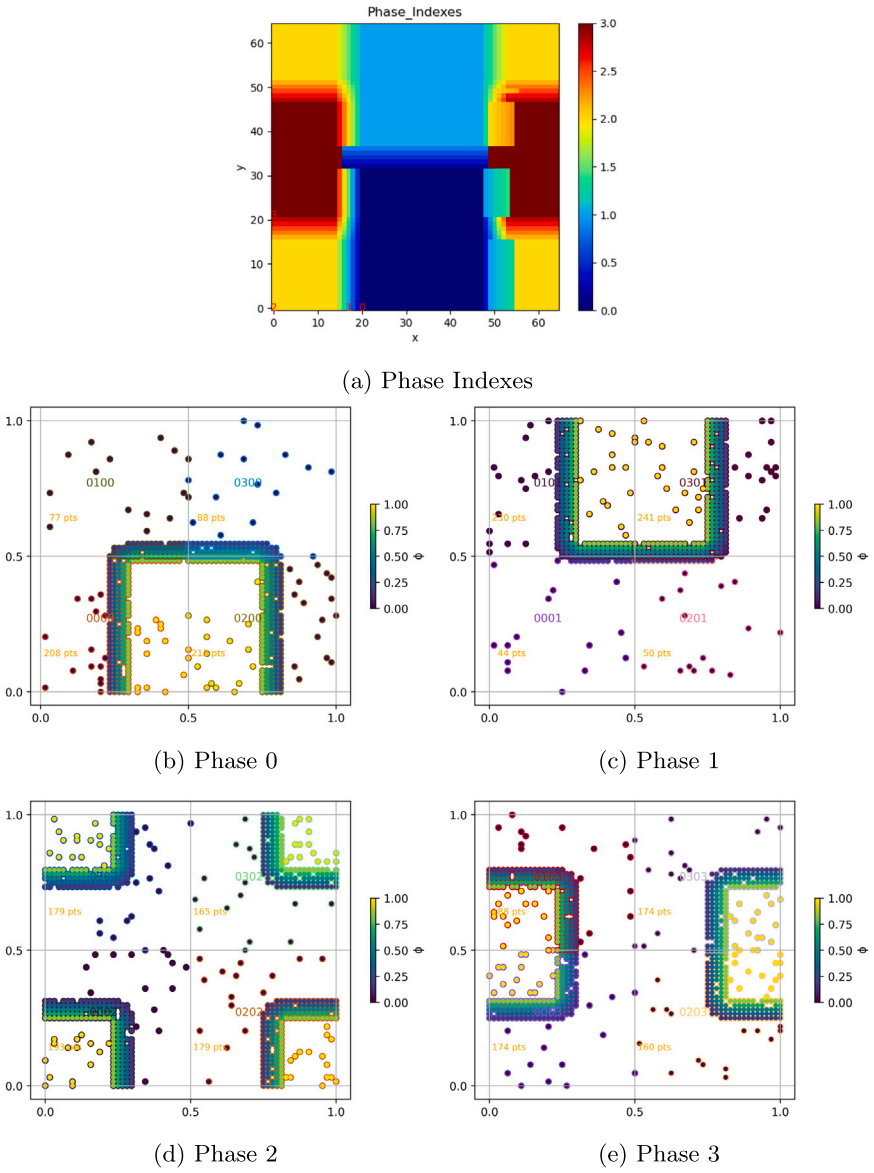


Fig. 7. Initial condition points for the different NNs handling the four phases in different batches. The corresponding mesh is given in the SM, section C. Reminder: the numbers on each batch correspond to the corresponding $PINN_i$ index (cf. Fig. 1). The IC points in each batch are surrounded by a circle with a common color for checking purposes.

1). Specifically, $N_{\text{batches min}}$=16 and N_{batches}=4 for the pyramidal approach, while $N_{\text{batches min}} = N_{\text{batches}}$ =4 for the second configuration. In Fig. 10(a), the impact of the optimization techniques is subsequently showcased: wheel of optimizers and the pyramidal training. The wheel of optimizers is activated with a period of 50 epochs; L-BFGS-B initially decreases the global loss from a magnitude of 10^{-1} to $\times 10^{-2}$ MSE, then a progressive decrease is achieved with subsequent rounds to below the threshold of 5×10^{-4} MSE. Then, Adam allows the transfer of learning from level 1 to level 2 of the pyramid, where the whole spatial domain is handled. Even if the global loss increases initially, it quickly falls below the target threshold (the 16 NNs converge together within 850 epochs), demonstrating the model's capacity to generalize the learnable solution to the whole domain. However, the non-adoption of such a strategy makes it hard to enforce the convergence of all the NNs, as seen in Fig. 10(b), and the global loss stagnates above the threshold even after 6000 epochs of training, using the same set of parameters as in Fig. 10(a) except for the non-activation of the pyramidal training. It is worth noting that such a comparison was subjected to excessive re-execution for repeatability with the same outputs; the 16 NNs failed to converge simultaneously without the activation of pyramidal training during the first time interval. It is also suggested that this finding is also attributed to two main impacts of pyramidal training: the progressive data augmentation basis of this concept, and the facilitation of addressing boundary conditions, as accurate boundary predictions are a direct consequence of good convergence within the domain. Additional details about the domain decomposition related to the pyramidal approach are provided in the SM, section D.

The black dashed line in Fig. 10c corresponds to the total mean loss, exhibiting oscillations that dip below the threshold (red line) for the enhancement of the next training interval. Here, the boundary condition loss is not visualized due to its discrete activation; however, its magnitude is noted to be in the range of 10^{-5}. It is noteworthy that all losses for all NNs should fall below this limit (not just the mean values). Previous observations regarding the behavior of the IC and PDE losses remain relatively valid in terms of the stabilization of losses across epochs. However, the declining aspect observed in the previous scenario is not evident here. This discrepancy could be attributed to the complexity of the triple junction evolution and dynamic interactions

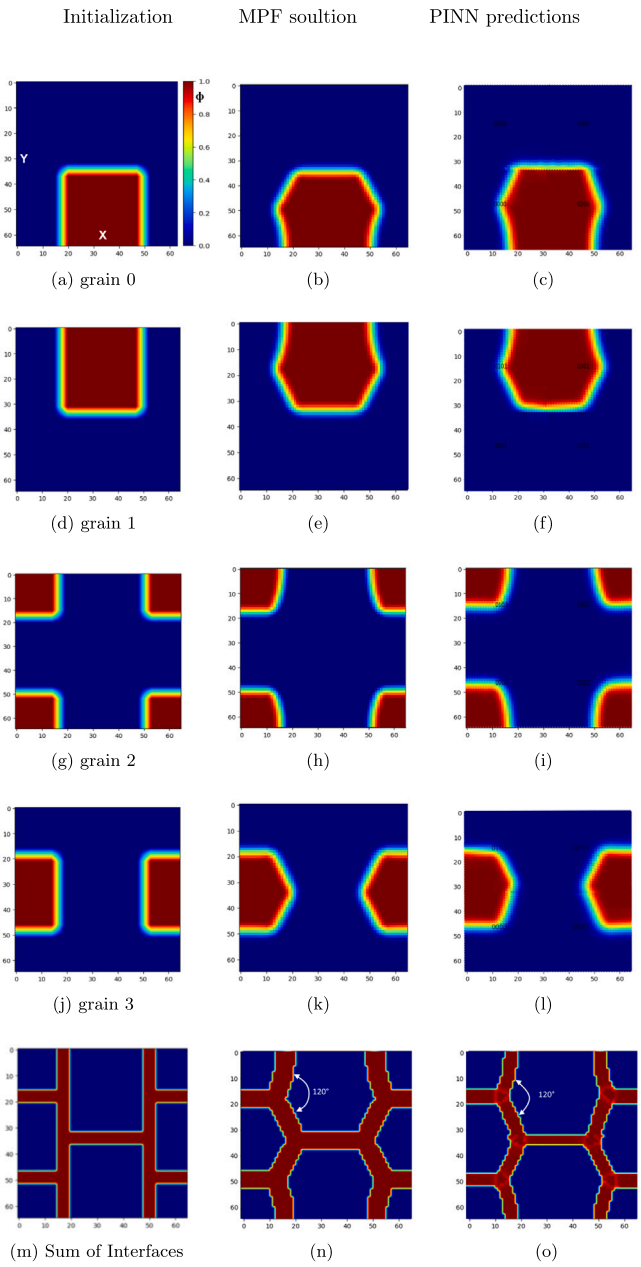


Fig. 8. Pure Phase-field solution (phase-field ϕ) against PINN predictions for a triple-junction problem involving four grains/phases in the system.

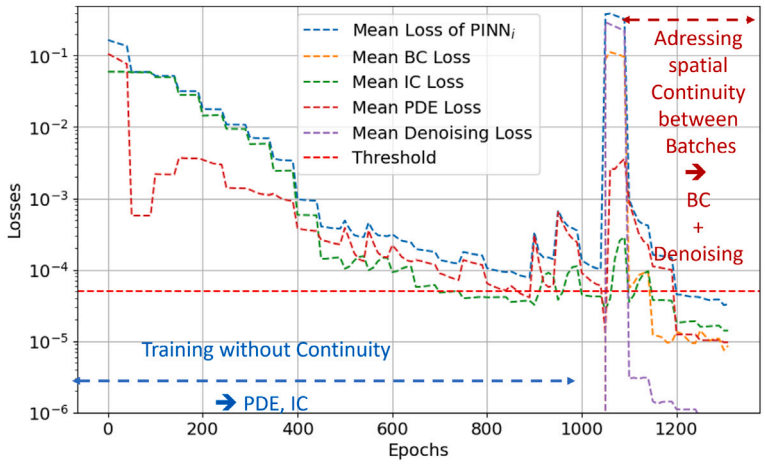
between different phases when compared to the single-grain shrinkage scenario. The sum loss (Eq. (16)) has the lowest magnitude (10^{-12}), and its constant values reflect the algorithm's capacity to dynamically correct predictions, ensuring adherence to the summation constraint. It is to note that the summation Eq. (2) is an intrinsic part of the MPF concept [49] and shown to stabilize the OpenPhase solutions, significantly [59].

Addressing nonlinearities in microstructure prediction.

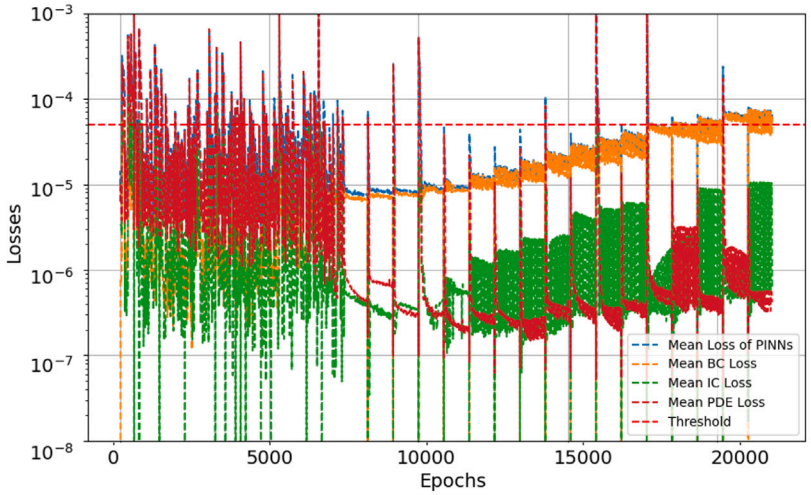
The current implementation showcases the effectiveness of this method in addressing nonlinearities such as in the MPF model, offering good predictions of microstructure evolution in different scenarios above. The ability to capture both curvature-driven and driving-force-driven interface dynamics is well demonstrated. Typical MPF simulations employ a time-stepping approach with a magnitude of up to 10^{-4} and a minimum of 1000 steps across the given scenarios that is to ensure computational stability. In contrast, applying PINNs is shown here to allow for the selection of much larger time-stepping up to 200 time steps, except for the traveling interface simulation where computational speed was a focal consideration. This difference stems from the fact that while the conventional MPF is limited due to the spatial resolution, the PINNs-MPF focuses on the overall microstructural patterns, substantiating its ability to capture underlying physics while maintaining stable computational schemes.

Tuning the model hyper-parameters.

One of the crucial points when dealing with PINNs is the tuning of the hyper-parameters. Here, any simplification made for hyper-parameters directly impacted convergence and can reduce the efforts for trial and error. In terms of the training dataset, this is automatically set, simplifying user inputs to the initialization, physical, and geometrical parameters. On the other hand, we have adopted major concepts from conventional MPF frameworks, namely the OpenPhase [64]. This includes considering our simulation domain heterogeneously, with a focus on the phase-field interfacial region, while parallelizing the handle of individual phase-field variables. This demonstrates that the



(a) Training on the first time interval t_0-t_1



(b) Training on the whole time domain)

Fig. 9. Evolution of the different losses (PDE, IC, BC, denoising and mean loss) for the four *PINN$_i$*s over the epochs for a grain shrink without driving force (for an interface width $\lambda = 7dx$). The plots are displayed in a logarithmic scale. In (b), values below 10^{-8} and above 10^{-3} are excluded from the plot for visualization purposes. Additionally, the denoising loss is omitted from the plot due to its low magnitude and fluctuating nature, which may affect the visualization of other losses.

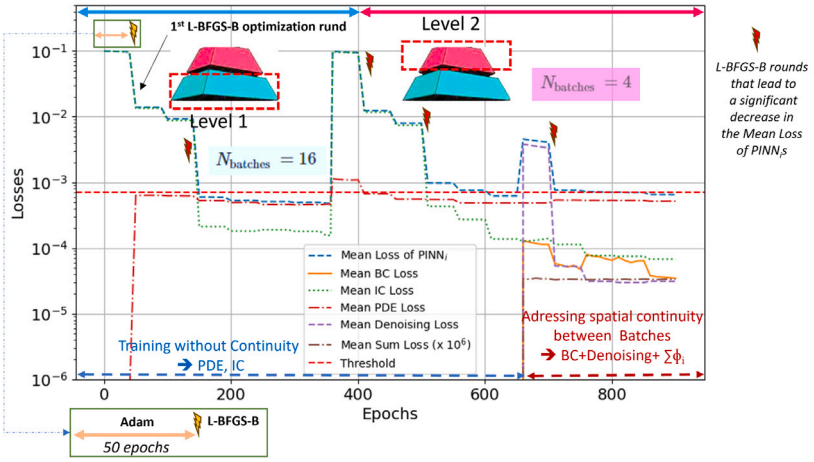
PINNs-MPF can successfully inherit optimization techniques already applied in classical computing approaches. Especially, the PINN resolution can be then assimilated as a multi-variable sequential learning, showcasing its adaptability and integration capabilities. The application of multi-networking and training by blocks demonstrates the feasibility of addressing diffuse interface problems with simplified hyper-parameters.

For specific challenges handling moving boundaries, the implementation of adaptive weights (gradient scaling algorithm) was unavoidable [39]. It is therefore worth noting that the loss terms in the implemented PINNs-MPF are by default left unweighted, i.e. $\lambda_{\text{pde}} = \lambda_{\text{bc}} = \lambda_{\text{ic}} = \lambda_{\sum\phi} = 1$ in Eq. (3). This reduces the complexity of the current framework and the need for meticulous tuning, especially when considering the diverse nature of loss terms in Eq. (3). Instead, we put a particular focus on the subdivision of the PINNs specifically to the MPF problem, ensuring efficient coupling between machine learning and optimizers. This is achieved through the proposed "wheel of optimizers", where the L-BFGS optimizer complements Adam to bring different key losses (e.g., PDE, IC, BC) to the same magnitude during training, that also help the model to handle all terms with equal importance. Moreover, this kind of algorithm could be useful to integrate in more complex scenarios. We note that our numerical experiments have indicated that the extrinsic losses can be weighted to optimize performance; for instance, reducing the weights for the BC loss to expedite training, and conversely, increasing the weighting of the phase summation loss to allocate more attention to it. However, the intrinsic losses (IC and PDE) cannot be weighted.

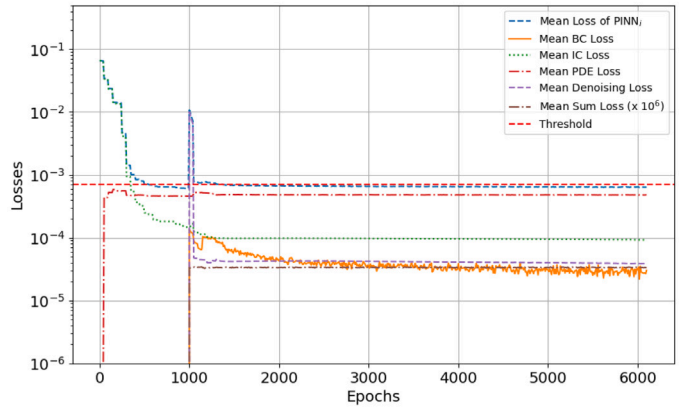
Interfacial attention for a latent microstructure representation.

The results of the conducted benchmark studies indicate that the global PINN solution in simulations involves a precise computation of solutions along interface regions throughout the entire domain, complemented by an extrapolation method to approximate solutions at the interface region. This hybrid approach proves beneficial for diffuse-interface problems using ML. In the triple junction scenario, the distributed '0 and 1' points, as depicted in Fig. 7, were employed to enhance the precision of the solution, particularly for the application of phase summation (Eq. (2)) and accurate addressing of boundary conditions between neighboring NNs. Similarly to the mono-phase benchmarks, it is possible to fully focus on interfacial regions within multi-phase simulations. First demonstrations of this are provided within Section E of the SM. This opens the way to resolve MPF simulations using NNs independently of the grid size, thus allowing for tackling more complex scenarios.

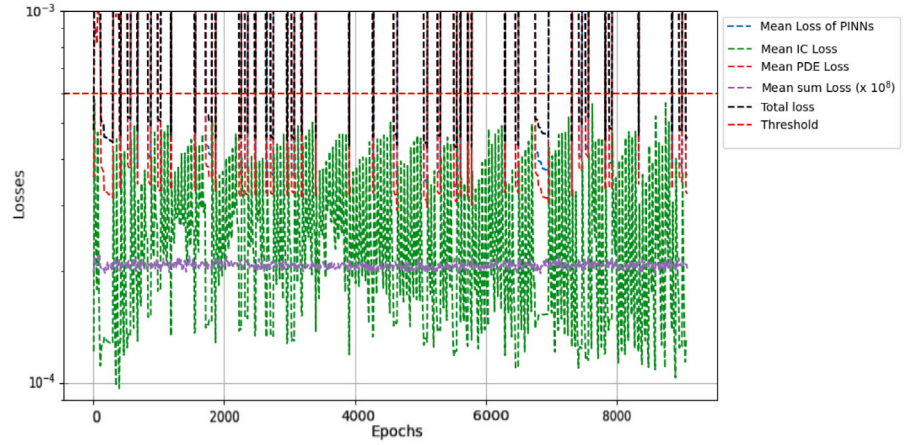
Dynamic correction mechanism for phase summation constraint.



(a) Training on the first time interval t_0-t_1 using the pyramidal approach.



(b) Training on the first time interval t_0-t_1 without the pyramidal approach.



(c) Training on the whole time domain.

Fig. 10. Evolution of the different losses for the 16 *PINN$_i$*s over the epochs for the triple junction scenario. The plot is displayed in a logarithmic scale.

Within the literature, a major limitation against the application of ML to resolve multi-phase problems is the phase summation criterion: Indeed, in previous works, a requirement for a bounding algorithm was obligatory as a correction step to the PINN prediction [44,47]. Meanwhile, in the current work, the applied method introduces a dynamic correction mechanism for phase predictions. This involves dividing the prediction of each neural network (NN) responsible for a specific phase by the sum of predictions from other NNs handling other phases within the same batch. This dynamic correction not only ensures continuous training but also suppresses the need for an external correction algorithm. Instead, the correction of phase summation is seamlessly integrated as an additional constraint loss term, enhancing the efficiency of the training process.

Exploring model complexity and impact of the optimization strategy.

The results about the additional numeric experiment are gathered in Table 5. Associated graphical illustration is provided in Fig. 11. The experiments revealed that while all architectures started with high and similar MSE values, the MSE decreased significantly after the first round of Adam optimization. The four models reach similar loss magnitude

Table 5
Evolution of MSE and model trainable parameters count for different architectures: optimization rounds with Adam-L-BFGS-Adam cycles. Comparisons are done at epoch 0, after first and second rounds.

HL × neurons/HL	Trainable parameters	MSE at epoch 0	MSE after round 1	MSE after round 2	Rounds required to converge[a]
6 x 64	105,605	1.396×10^{1}	1.387×10^{-1}	1.270^{-2}	5
6 x 128	416,005	1.33×10^{1}	2.712×10^{-1}	1.29×10^{-4}	3
6 x 256	1,651,205	1.36×10^{1}	1.87×10^{-1}	1.387×10^{-1}	$\succ 10$
6 x 512	6,579,205	1.351×10^{1}	7.517×10^{-1}	1.723×10^{-1}	$\succ 10$

[a] It is worth reminding that convergence signifies that the global loss of the model has fallen below the threshold and the current time interval has been managed.

after the first round. Subsequent L-BFGS optimization further reduced the MSE for the 6 × 64 and 6 × 128 architectures, with the 6 × 128 model attaining the best overall MSE of 1.29×10^{-4}. Larger architectures like 6 × 256 and 6 × 512 failed to converge within the specified rounds, potentially due to overfitting or optimization challenges arising from their high parameter counts. The 6 × 128 architecture struck an optimal balance between model complexity and generalization ability, benefiting from the combined Adam and L-BFGS optimization strategy. While larger models had more parameters, they did not necessarily perform better and could be computationally expensive. This numerical experiment reveals a solid correlation between the model's parameter count, representing its complexity, and the impact of multi-networking. For instance, the 6 × 256 configuration, yielding 1,651,205 trainable parameters, challenges training when employing four NNs. This means that the "wheel of optimizers" approach fails to achieve convergence, marking the scalability limit. Conversely, a reduced parameter count of 1,414,417 enables training 16 NNs (16 x (6 HL × 128 neurons)) simultaneously to effectively handle four phases (c.f. the triple junction benchmark), confirming its suitability for upscaling the simulation domains. Moreover, even higher parallelism is feasible through pyramidal training, where initially 64 NNs are initialized, but only 16 are selected for further training using the basket of PINNs concept. That implies that using moderately-sized networks through the multi-networking context (the extended subdomain decomposition technique) allows to efficiently alternate optimizers (the wheel of optimizers), while the transfer of learning facilitates the transition between time intervals. This underscores the rationale behind initially selecting a subset of PINN techniques, while the second subset (AMFO, phase correction, and vertical optimization) has proven crucial in prior MPF method studies and the current work. Pyramidal training and the comprehensive set of PINNs emerge as essential for progressively transferring learning to a unified domain when dealing with massive domain decomposition, thereby making the framework ready for up scaling, as hereafter detailed.

Transfer of learning.

In the context of learning transfer, our framework demonstrates the feasibility of hierarchically transferring knowledge within subdomain decomposition. Through multi-networking and full parallel training, it employs a pyramidal training approach, categorized as 'Multiple to Multiple' transfer. This allows the transfer of learning, for example (but not limited to), from 4 NNs to 4 others, 16 to 16, 16 to 4, etc. From a technical standpoint, achieving 'Multiple to Single' transfer of learning is possible, where knowledge is transferred from multiple NNs to a single NN and vice versa. However, our first trials showcased that training a single NN revealed challenges, leading to optimization failures. The synergy between ML and optimization algorithms is crucial for effectively exploiting patterns, but this task becomes challenging with large NNs. Multi-networking and pyramidal training until reaching a reasonable number of NNs proves efficient in managing such complexities. Nevertheless, we provide within the actual code repository a demonstration of this 'Multiple to Single' transfer, denoted as '$PINN_i s$ to *Master*' transfer, that could be of interest for other physical applications. Additional details are provided in the SM, section F.

Exploring more ways for improvement of the framework.

In the current development phase, one aspect to consider is selecting the appropriate threshold to transition between time intervals in the full-discrete resolution. This threshold was found to be strictly related to each simulation and subjected to trial and error. For the magnitude of 10^{-5} in single-phase benchmarks, it was slightly raised when handling the multiphase field (MPF) scenario to ensure a compromise between the speed of the training and the convergence of the solution. One possibility to deal with this limitation is to integrate an additional loss term related to the variation in the energy of the system. If the energy residual (and its derivative) reaches a steady state, it is then a physical indication that the system reaches its equilibrium within this time interval, and then the transition could be ensured [88,89]. Theoretical formulations and graphical illustrations related to the utility of integrating energy loss to enhance training are provided in the SM, section B.

A potential point of improvement for the current PINNs-MPF framework is the integration of sequential training. Indeed, optimizing the evolution of losses related to the triple junction motion could be achieved if the model captures the temporal behavior of the solution. The flexibility of discrete resolution under the current implementation eases the integration of sequential learning through Recurrent Neural Networks. Optimizing runtimes against theoretical solutions or existing literature is beyond the scope of this study, as PINNs are inherently more time-consuming than classical approaches. Enhancing PINN solvers is a work in progress [7,90] and studies on the use of PINNs for solving multi-phase problems remain uncommon [39] . Therefore, the first goal of this study is to prioritize solution fidelity, training continuity, and efficient resource distribution. This approach is similar to recent studies focusing on multiphase phenomena, particularly in fluid dynamics. For instance, Haghighat et al. [39] have concentrated on the methodology and challenges associated with training PINNs for coupled flow and deformation in porous media. Zheng et al. [44] have aimed to predict the sequential motions of flow simulations in a discontinuous manner, alternating between PINN training and MCBOM corrections. Amini et al. [41] have focused on validating the inverse modeling approach targeting nonisothermal multiphase poromechanics. To conduct running time studies on these multi-phase field frameworks, inspiration could be taken from the study of Shukla et al. [72]. Among other proposals, leveraging multiple GPUs and nodes and adopting a hybrid programming model could be beneficial.

6. Conclusion and outlook

PINNs-MPF framework has been presented to resolve MPF simulation of interface dynamics using capacities in physics-informed NNs. Our approach has been to develop several novel interconneted novel concepts to address and benchmark the most central requirements for a reliable reproduction of MPF simulations, namely, curvature- and driving-force-driven interface motions and the evolution and equilibrium of a triple junction. In particular, the nonlinear evolution of curved interfaces and the effect of interface width on the interface dynamics were captured, demonstrating the capability of this framework in dealing with diffuse interfaces. The simulation of the triple

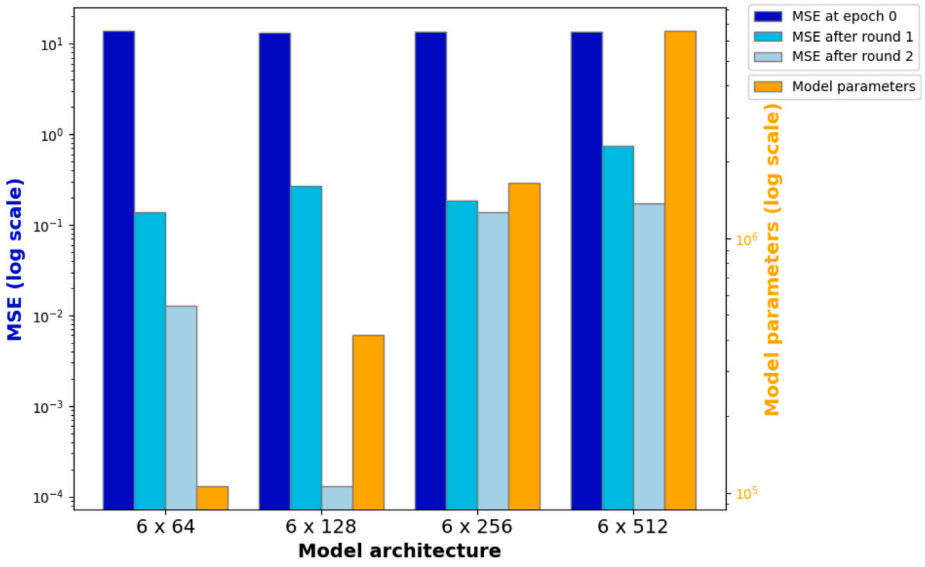


Fig. 11. Evolution of MSEs and model parameters count for different architectures as variants of the benchmark 2.

junction established that the application of the sum constraint as a loss term and in combination with a parallelization scheme works precisely and efficiently. The rationale behind the current development has been to demonstrate the feasibility of PINNs in capturing key features of microstructure evolution, before increasing complexities. This ensures a well-prepared transition to more advanced dimensions while maintaining the robustness and reliability of the methodology. All benchmark simulations are conducted in a single execution without the need for post-correction algorithms, such as intercalated phase correction algorithms. Various advanced techniques were encompassed in the PINNs-MPF framework, including training optimization, extended domain decomposition, and boundary condition propagation. These techniques have yielded accurate predictions and efficient training, establishing a robust foundation for future advancements in this research area.

Potential avenues for further exploration include developing more unified solutions through sequential learning, conducting comparative studies with other PINN approaches in the literature, and integrating energy-based optimizations, which is the key component of the MPF Method. As a next immediate step, it is deemed necessary to enhance our framework in order to address large-scale MPF simulations. Following the current philosophy, it is therefore needed to extend the capability of the developed framework to address more intricate scenarios involving a greater number of phases, larger grids, and three-dimensional dimensions. As the difficulty of the scenarios gradually increases, it is crucial to carefully address related issues when using machine learning to predict solutions, such as managing singularities, mitigating inaccurate predictions, and optimizing computation times.

CRediT authorship contribution statement

Seifallah Elfetni: Writing – original draft, Visualization, Software, Methodology, Formal analysis, Conceptualization. **Reza Darvishi Kamachali:** Writing – review & editing, Conceptualization, Methodology, Formal analysis, Validation, Supervision, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

RDK gratefully acknowledges financial support from the German Research Foundation (DFG), Germany within projects *DA 1655/2-1* (Heisenberg program) and *DA 1655/5-1*.

Data availability

The PINNs-MPF repository is available on GitHub at the following link: https://github.com/SFETNI/PINNs_MPF

The verified code can also be found as an oceancode capsule in [84].

Appendix A. Supplementary data

Additional results and discussions as well as a video presentation of the PINNs-MPF framework are presented in the Supplementary Material attached.

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.enganabound.2025.106200.

Data availability

The PINNs-MPF repository is available on GitHub at the following link:

https://github.com/SFETNI/PINNs_MPF

The verified code can also be found as an oceancode capsule in [84].

References

[1] Hart Gus LW, Mueller Tim, Toher Cormac, Curtarolo Stefano. Machine learning for alloys. Nat Rev Mater 2021;6(8):730–55.

[2] Choudhary Kamal, DeCost Brian, Chen Chi, Jain Anubhav, Tavazza Francesca, Cohn Ryan, et al. Recent advances and applications of deep learning methods in materials science. Npj Comput Mater 8(1). http://dx.doi.org/10.1038/s41524-022-00734-6.

[3] Menon Sarath, Lysogorskiy Yury, Knoll Alexander LM, Leimeroth Niklas, Poul Marvin, Qamar Minaam, Janssen Jan, Mrovec Matous, Rohrer Jochen, Albe Karsten, et al. From electrons to phase diagrams with machine learning potentials using pyiron based automated workflows. npj Comput Mater 2024;10(1):261.

[4] Zhang Lei, Stricker Markus. MatNexus: A comprehensive text mining and analysis suite for materials discovery. SoftwareX 2024;26:101654.

[5] Schmidt Jonathan, Cerqueira Tiago FT, Romero Aldo H, Loew Antoine, Jäger Fabian, Wang Hai-Chen, et al. Improving machine-learning models in materials science through large datasets. Mater Today Phys 2024;48:101560.

[6] Dösinger Christoph, Hammerschmidt Thomas, Peil Oleg, Scheiber Daniel, Romaner Lorenz. Descriptors based on the density of states for efficient machine learning of grain-boundary segregation energies. Comput Mater Sci 2025;247:113493.

[7] Cuomo Salvatore, Di Cola Vincenzo Schiano, Giampaolo Fabio, Rozza Gianluigi, Raissi Maziar, Piccialli Francesco. Scientific machine learning through physics–informed neural networks: Where we are and what's next. J Sci Comput 2022;92(3):88. http://dx.doi.org/10.1007/s10915-022-01939-z.

[8] Haghighat Ehsan, Raissi Maziar, Moure Adrian, Gomez Hector, Juanes Ruben. A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics. Comput Methods Appl Mech Engrg 2021;379:113741. http://dx.doi.org/10.1016/j.cma.2021.113741.

[9] Ramuhalli P, Udpa L, Udpa SS. Finite-element neural networks for solving differential equations. IEEE Trans Neural Netw 2005;16(6):1381–92. http://dx.doi.org/10.1109/TNN.2005.857945.

[10] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys 2019;378:686–707. http://dx.doi.org/10.1016/j.jcp.2018.10.045, URL: https://linkinghub.elsevier.com/retrieve/pii/S0021999118307125.

[11] Yang Yibo, Perdikaris Paris. Adversarial uncertainty quantification in physics-informed neural networks. J Comput Phys 2019;394:136–52. http://dx.doi.org/10.1016/j.jcp.2019.05.027, URL: https://www.sciencedirect.com/science/article/pii/S0021999119303584.

[12] Zhang Dongkun, Lu Lu, Guo Ling, Karniadakis George Em. Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems. J Comput Phys 2019;397:108850. http://dx.doi.org/10.1016/j.jcp.2019.07.048.

[13] Prakhar Sharma Michelle Tindall, Nithiarasu Perumal. Hyperparameter selection for physics-informed neural networks (PINNs) – Application to discontinuous heat conduction problems. Numer Heat Transfer B 2023;1–15. http://dx.doi.org/10.1080/10407790.2023.2264489.

[14] Zobeiry Navid, Humfeld Keith D. A physics-informed machine learning approach for solving heat transfer equation in advanced manufacturing and engineering applications. Eng Appl Artif Intell 2021;101:104232. http://dx.doi.org/10.1016/j.engappai.2021.104232.

[15] Diao Yu, Yang Jianchuan, Zhang Ying, Zhang Dawei, Du Yiming. Solving multi-material problems in solid mechanics using physics-informed neural networks based on domain decomposition technology. Comput Methods Appl Mech Engrg 2023;413:116120. http://dx.doi.org/10.1016/j.cma.2023.116120, URL: https://www.sciencedirect.com/science/article/pii/S004578252300244X.

[16] Henkes Alexander, Wessels Henning, Mahnken Rolf. Physics informed neural networks for continuum micromechanics. Comput Methods Appl Mech Engrg 2022;393:114790. http://dx.doi.org/10.1016/j.cma.2022.114790, URL: https://www.sciencedirect.com/science/article/pii/S0045782522001268.

[17] Guo Xin-Yu, Fang Sheng-En. Structural parameter identification using physics-informed neural networks. Measurement 2023;220:113334. http://dx.doi.org/10.1016/j.measurement.2023.113334.

[18] Bai Jinshuai, Jeong Hyogu, Batuwatta-Gamage CP, Xiao Shusheng, Wang Qingxia, Rathnayaka CM, et al. An introduction to programming physics-informed neural network-based computational solid mechanics. Int J Comput Methods 2023;20(10):2350013. http://dx.doi.org/10.1142/S0219876223500135.

[19] Goswami Somdatta, Anitescu Cosmin, Chakraborty Souvik, Rabczuk Timon. Transfer learning enhanced physics informed neural network for phase-field modeling of fracture. Theor Appl Fract Mech 2020;106:102447. http://dx.doi.org/10.1016/j.tafmec.2019.102447, URL: https://www.sciencedirect.com/science/article/pii/S016784421930357X.

[20] Khorrami MS, Mianroodi JR, Siboni NH, et al. An artificial neural network for surrogate modeling of stress fields in viscoplastic polycrystalline materials. Npj Comput Mater 2023;9:37. http://dx.doi.org/10.1038/s41524-023-00991-z.

[21] Mazhar Farrukh, Javed Ali, Altinkaynak Atakan. A novel artificial neural network-based interface coupling approach for partitioned fluid–structure interaction problems. Eng Anal Bound Elem 2023;151:287–308. http://dx.doi.org/10.1016/j.enganabound.2023.02.022.

[22] Luo Zhiqiang, Yan Chengzeng, Ke Wenhui, Wang Tie, Xiao Mingzhao. A surrogate model based on deep convolutional neural networks for solving deformation caused by moisture diffusion. Eng Anal Bound Elem 2023;157:353–73. http://dx.doi.org/10.1016/j.enganabound.2023.09.009.

[23] Cao Yuli, Xu Ruina, Jiang Peixue. Physics-informed machine learning based RANS turbulence modeling convection heat transfer of supercritical pressure fluid. Int J Heat Mass Transfer 2023;201:123622. http://dx.doi.org/10.1016/j.ijheatmasstransfer.2022.123622.

[24] Luo Shirui, Vellakal Madhu, Koric Seid, Kindratenko Volodymyr, Cui Jiahuan. Parameter identification of RANS turbulence model using physics-embedded neural network. In: Jagode Heike, Anzt Hartwig, Juckeland Guido, Ltaief Hatem, editors. High performance computing. Cham: Springer International Publishing; 2020, p. 137–49.

[25] Hanrahan S, Kozul M, Sandberg RD. Studying turbulent flows with physics-informed neural networks and sparse data. Int J Heat Fluid Flow 2023;104:109232. http://dx.doi.org/10.1016/j.ijheatfluidflow.2023.109232, URL: https://www.sciencedirect.com/science/article/pii/S0142727X23001315.

[26] Chen Haotian, Batchelor-McAuley Christopher, Kätelhön Enno, Elliott Joseph, Compton Richard G. A critical evaluation of using physics-informed neural networks for simulating voltammetry: Strengths, weaknesses and best practices. J Electroanal Chem 2022;925:116918. http://dx.doi.org/10.1016/j.jelechem.2022.116918.

[27] Hofmann Tobias, Hamar Jacob, Rogge Marcel, Zoerr Christoph, Erhard Simon, Schmidt Jan Philipp. Physics-informed neural networks for state of health estimation in lithium-ion batteries. J Electrochem Soc 2023;170(9):090524. http://dx.doi.org/10.1149/1945-7111/acf0ef.

[28] Han Jiang-Xia, Xue Liang, Wei Yun-Sheng, Qi Ya-Dong, Wang Jun-Lei, Liu Yue-Tian, et al. Physics-informed neural network-based petroleum reservoir simulation with sparse data using domain decomposition. Pet Sci 2023;20(6):3450–60. http://dx.doi.org/10.1016/j.petsci.2023.10.019, URL: https://www.sciencedirect.com/science/article/pii/S1995822623002947.

[29] Priyadarshi Gaurav, Murali Cheepurupalli, Agarwal Sumit, Naik B Kiran. Parametric investigation and optimization of phase change material-based thermal energy storage integrated desiccant coated energy exchanger through physics informed neural networks oriented deep learning approach. J Energy Storage 2024;80:110231. http://dx.doi.org/10.1016/j.est.2023.110231, URL: https://www.sciencedirect.com/science/article/pii/S2352152X23036307.

[30] D. Jagtap Ameya, Em Karniadakis George. Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. Commun Comput Phys 2020;28(5):2002–41. http://dx.doi.org/10.4208/cicp.OA-2020-0164.

[31] Zhang Lingxiao, Li Xinxiang. Deep Interface Alternation Method (DIAM) based on domain decomposition for solving elliptic interface problems. Eng Anal Bound Elem 2024;168:105905. http://dx.doi.org/10.1016/j.enganabound.2024.105905.

[32] Kharazmi Ehsan, Zhang Zhongqiang, Karniadakis George EM. hp-VPINNs: Variational physics-informed neural networks with domain decomposition. Comput Methods Appl Mech Engrg 2021;374:113547. http://dx.doi.org/10.1016/j.cma.2020.113547.

[33] Meng Xuhui, Li Zhen, Zhang Dongkun, Karniadakis George Em. PPINN: Parareal physics-informed neural network for time-dependent PDEs. Comput Methods Appl Mech Engrg 2020;370:113250. http://dx.doi.org/10.1016/j.cma.2020.113250, URL: https://www.sciencedirect.com/science/article/pii/S0045782520304357.

[34] Penwarden Michael, Jagtap Ameya D, Zhe Shandian, Karniadakis George Em, Kirby Robert M. A unified scalable framework for causal sweeping strategies for Physics-Informed Neural Networks (PINNs) and their temporal decompositions. J Comput Phys 2023;493:112464. http://dx.doi.org/10.1016/j.jcp.2023.112464.

[35] Krishnapriyan A, Gholami A, Zhe S, Kirby R, Mahoney MW. Characterizing possible failure modes in physics-informed neural networks. In: Advances in neural information processing systems, vol. 34. 2021, p. 26548–60.

[36] Bihlo Alex, Popovych Roman O. Physics-informed neural networks for the shallow-water equations on the sphere. J Comput Phys 2022;456:111024. http://dx.doi.org/10.1016/j.jcp.2022.111024.

[37] Rojas Carlos JG, Boldrini Jos L, Bittencourt Marco L. Parameter identification for a damage phase field model using a physics-informed neural network. Theor Appl Mech Lett 2023;13(3):100450. http://dx.doi.org/10.1016/j.taml.2023.100450.

[38] Zhang Wen, Li Jian. The robust physics-informed neural networks for a typical fourth-order phase field model. Comput Math Appl 2023;140:64–77. http://dx.doi.org/10.1016/j.camwa.2023.03.016.

[39] Haghighat Ehsan, Amini Danial, Juanes Ruben. Physics-informed neural network simulation of multiphase poroelasticity using stress-split sequential training. Comput Methods Appl Mech Engrg 2022;397:115141.

[40] Zhang Zhao, Yan Xia, Liu Piyang, Zhang Kai, Han Renmin, Wang Sheng. A physics-informed convolutional neural network for the simulation and prediction of two-phase Darcy flows in heterogeneous porous media. J Comput Phys 2023;477:111919. http://dx.doi.org/10.1016/j.jcp.2023.111919.

[41] Amini Danial, Haghighat Ehsan, Juanes Ruben. Inverse modeling of non-isothermal multiphase poromechanics using physics-informed neural networks. J Comput Phys 2023;490:112323. http://dx.doi.org/10.1016/j.jcp.2023.112323.

[42] Wight Colby L, Zhao Jia. Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks. 2020, arXiv:2007.04542, [math.NA].

[43] Singh Anjali, Sinha Rajen Kumar. SS-DNN: A hybrid strang splitting deep neural network approach for solving the Allen–Cahn equation. Eng Anal Bound Elem 2024;169:105944. http://dx.doi.org/10.1016/j.enganabound.2024.105944.

[44] Zheng Haoyang, Huang Ziyang, Lin Guang. A physics-constrained neural network for multiphase flows. Phys Fluids 2022;34(10). http://dx.doi.org/10.1063/5.0111275.

[45] Zhu Yinhao, Zabaras Nicholas, Koutsourelakis Phaedon-Stelios, Perdikaris Paris. Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data. J Comput Phys 2019;394:56–81. http://dx.doi.org/10.1016/j.jcp.2019.05.024.

[46] Arzani Amirhossein, Cassel Kevin W, D'Souza Roshan M. Theory-guided physics-informed neural networks for boundary layer problems with singular perturbation. J Comput Phys 2023;473:111768. http://dx.doi.org/10.1016/j.jcp.2022.111768.

[47] Huang Ziyang, Lin Guang, Ardekani Arezoo M. A consistent and conservative volume distribution algorithm and its applications to multiphase flows using phase-field models. Int J Multiph Flow 2021;142:103727. http://dx.doi.org/10.1016/j.ijmultiphaseflow.2021.103727.

[48] Chen Long-Qing. Phase-field models for microstructure evolution. Annu Rev Mater Res 2002;32:113–40.

[49] Steinbach Ingo. Phase-field models in materials science. Modelling Simul Mater Sci Eng 2009;17:073001.

[50] Darvishi Kamachali Reza, Kim Se-Jong, Steinbach Ingo. Texture evolution in deformed AZ31 magnesium sheets: Experiments and phase-field study. Comput Mater Sci 2015;104:193–9.

[51] Shi Rongpei, McAllister Donald P, Zhou Ning, Detor Andrew J, DiDomizio Richard, Mills Michael J, et al. Growth behavior of γ'/γ''coprecipitates in Ni-Base superalloys. Acta Mater 2019;164:220–36.

[52] Wang Kai, De Souza Roger A, Peng Xiang-Long, Merkle Rotraut, Rheinheimer Wolfgang, Albe Karsten, et al. A defect-chemistry-informed phase-field model of grain growth in oxide electroceramics. 2024, arXiv preprint arXiv:2407.17650.

[53] Darvishi Kamachali Reza, Schwarze Christian. Inverse ripening and rearrangement of precipitates under chemomechanical coupling. Comput Mater Sci 2017;130:292–6.

[54] Schwarze Christian, Darvishi Kamachali Reza, Kühbach Markus, Mießen Christian, Tegeler Marvin, Barrales-Mora Luis, et al. Computationally efficient phase-field simulation studies using RVE sampling and statistical analysis. Comput Mater Sci 2018;147:204–16.

[55] Kadirvel Kamalnath, Fraser Hamish L, Wang Yunzhi. Microstructural design via spinodal-mediated phase transformation pathways in high-entropy alloys (HEAs) using phase-field modelling. Acta Mater 2023;243:118438.

[56] Schiedung Raphael, Darvishi Kamachali Reza, Steinbach Ingo, Varnik Fathollah. Multi-phase-field model for surface and phase-boundary diffusion. Phys Rev E 2017;96(1):012801.

[57] Ruan Hui, Peng Xiang-Long, Yang Yangyiwei, Gross Dietmar, Xu Bai-Xiang. Phase-field ductile fracture simulations of thermal cracking in additive manufacturing. J Mech Phys Solids 2024;191:105756.

[58] Darvishi Kamachali Reza, Schwarze Christian, Lin Mingxuan, Diehl Martin, Shanthraj Pratheek, Prahl Ulrich, et al. Numerical benchmark of phase-field simulations with elastic strains: precipitation in the presence of chemo-mechanical coupling. Comput Mater Sci 2018;155:541–53.

[59] Tegeler Marvin, Shchyglo Oleg, Darvishi Kamachali Reza, Monas Alexander, Steinbach Ingo, Sutmann Godehard. Parallel multiphase field simulations with OpenPhase. Comput Phys Comm 2017;215:173–87.

[60] Wang C, Tan XP, Tor SB, Lim CS. Machine learning in additive manufacturing: State-of-the-art and perspectives. Addit Manuf 2020;36:101538. http://dx.doi.org/10.1016/j.addma.2020.101538.

[61] Parsazadeh Mohammad, Sharma Shashank, Dahotre Narendra. Towards the next generation of machine learning models in additive manufacturing: A review of process dependent material evolution. Prog Mater Sci 2023;135:101102. http://dx.doi.org/10.1016/j.pmatsci.2023.101102.

[62] Qiao Ling, Liu Yong, Zhu Jingchuan. A focused review on machine learning aided high-throughput methods in high entropy alloy. J Alloys Compd 2021;877:160295. http://dx.doi.org/10.1016/j.jallcom.2021.160295, URL: https://www.sciencedirect.com/science/article/pii/S0925838821017047.

[63] Rao Ziyuan, Tung Po-Yen, Xie Ruiwen, Wei Ye, Zhang Hongbin, Ferrari Alberto, et al. Machine learning–enabled high-entropy alloy discovery. Science 2022;378(6615):78–85. http://dx.doi.org/10.1126/science.abo4940.

[64] Darvishi Kamachali Reza. Grain boundary motion in polycrystalline materials (Ph.D. thesis), Univ.-Bibliothek; 2013.

[65] Secci Daniele, Godoy Vanessa A, Gómez-Hernández J Jaime. Physics-Informed Neural Networks for solving transient unconfined groundwater flow. Comput Geosci 2024;182:105494. http://dx.doi.org/10.1016/j.cageo.2023.105494.

[66] Montes de Oca Zapiain D, Stewart JA, Dingreville R. Accelerating phase-field-based microstructure evolution predictions via surrogate models trained by machine learning methods. Npj Comput Mater 2021;7:3. http://dx.doi.org/10.1038/s41524-020-00471-8.

[67] Hu C, Martin S, Dingreville R. Accelerating phase-field predictions via recurrent neural networks learning the microstructure evolution in latent space. Comput Methods Appl Mech Engrg 2022;397:115128. http://dx.doi.org/10.1016/j.cma.2022.115128.

[68] Oommen V, Shukla K, Goswami S, et al. Learning two-phase microstructure evolution using neural operators and autoencoder architectures. Npj Comput Mater 2022;8:190. http://dx.doi.org/10.1038/s41524-022-00876-7.

[69] Fetni Seifallah, Pham Thinh Quy Duc, Hoang Truong Vinh, Tran Hoang Son, Duchêne Laurent, Tran Xuan-Van, et al. Capabilities of auto-encoders and principal component analysis of the reduction of microstructural images; application on the acceleration of phase-field simulations. Comput Mater Sci 2023;216:111820. http://dx.doi.org/10.1016/j.commatsci.2022.111820.

[70] Chen Zhaolin, Lai Siu-Kai, Yang Zhichun. AT-PINN: Advanced time-marching physics-informed neural network for structural vibration analysis. Thin-Walled Struct 2024;196:111423. http://dx.doi.org/10.1016/j.tws.2023.111423, URL: https://www.sciencedirect.com/science/article/pii/S0263823123009011.

[71] Wang Sifan, Sankaran Shyam, Perdikaris Paris. Respecting causality for training physics-informed neural networks. Comput Methods Appl Mech Engrg 2024;421:116813. http://dx.doi.org/10.1016/j.cma.2024.116813, URL: https://www.sciencedirect.com/science/article/pii/S0045782524000690.

[72] Shukla Khemraj, Jagtap Ameya D, Karniadakis George Em. Parallel physics-informed neural networks via domain decomposition. J Comput Phys 2021;447:110683. http://dx.doi.org/10.1016/j.jcp.2021.110683.

[73] Hüter C, Yin X, Vo T, Braun S. A pragmatic dataset augmentation approach for transformation temperature prediction in steels. Comput Mater Sci 2020;176:109488. http://dx.doi.org/10.1016/j.commatsci.2019.109488.

[74] Carpenter Linda L, Yasmin Sarah, Price Lawrence H. A double-blind, placebo-controlled study of antidepressant augementation with mirtazapine. Biol Psychiatry 2002;51(2):183–8. http://dx.doi.org/10.1016/S0006-3223(01)01262-8, URL: https://www.sciencedirect.com/science/article/pii/S0006322301012628.

[75] Li Yibao, Kim Junseok. Phase-field simulations of crystal growth with adaptive mesh refinement. Int J Heat Mass Transfer 2012;55(25):7926–32. http://dx.doi.org/10.1016/j.ijheatmasstransfer.2012.08.009, URL: https://www.sciencedirect.com/science/article/pii/S0017931012006308.

[76] Gupta Abhinav, Krishnan U Meenu, Mandal Tushar Kanti, Chowdhury Rajib, Nguyen Vinh Phu. An adaptive mesh refinement algorithm for phase-field fracture models: Application to brittle, cohesive, and dynamic fracture. Comput Methods Appl Mech Engrg 2022;399:115347. http://dx.doi.org/10.1016/j.cma.2022.115347, URL: https://www.sciencedirect.com/science/article/pii/S0045782522004339.

[77] Xu Wenqiang, Li Yu, Li Hanzhang, Qiang Sheng, Zhang Chengpeng, Zhang Caihong. Multi-level adaptive mesh refinement technique for phase-field method. Eng Fract Mech 2022;276:108891. http://dx.doi.org/10.1016/j.engfracmech.2022.108891, URL: https://www.sciencedirect.com/science/article/pii/S0013794422006099.

[78] Freddi F, Mingazzi L. Adaptive mesh refinement for the phase field method: A FEniCS implementation. Appl Eng Sci 2023;14:100127. http://dx.doi.org/10.1016/j.apples.2023.100127, URL: https://www.sciencedirect.com/science/article/pii/S266649682300002X.

[79] Bijaya Ananya, Gupta Abhinav, Krishnan U Meenu, Chowdhury Rajib. A multilevel adaptive mesh scheme for efficient simulation of thermomechanical phase-field fracture. J Eng Mech 2023;150(6). http://dx.doi.org/10.1061/JENMDT.EMENG-7480.

[80] Wu Chenxi, Zhu Min, Tan Qinyang, Kartha Yadhu, Lu Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. Comput Methods Appl Mech Engrg 2023;403:115671. http://dx.doi.org/10.1016/j.cma.2022.115671, URL: https://www.sciencedirect.com/science/article/pii/S0045782522006260.

[81] Jagtap Ameya D, Kawaguchi Kenji, Karniadakis George Em. Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. J Comput Phys 2020;404:109136. http://dx.doi.org/10.1016/j.jcp.2019.109136.

[82] Li Shirong, Feng Xinlong. Dynamic weight strategy of physics-informed neural networks for the 2D Navier–Stokes equations. Entropy 2022;24(9):1254. http://dx.doi.org/10.3390/e24091254.

[83] Daw Arka, Bu Jie, Wang Sifan, Perdikaris Paris, Karpatne Anuj. Mitigating propagation failures in physics-informed neural networks using retain-resample-release (r3) sampling. In: ICML'23: proceedings of the 40th international conference on machine learning. 2023, p. 7264–302.

[84] Elfetni Seifallah, Kamachali Reza Darvishi. PINNs-MPF: a physics informed neural network for multi phase field simulations [source code]. 2024, http://dx.doi.org/10.24433/CO.4820496.v1, OceanCode Repository.

[85] Weinan E, Yu Bing. The deep ritz method: A deep learning-based numerical algorithm for solving variational problems. Commun Math Stat 2018;6:1–12. http://dx.doi.org/10.1007/s40304-018-0127-z, arXiv: 1710.00211.

[86] Wang Lei, Darvishi Kamachali Reza. Incorporating elasticity into CALPHAD-informed density-based grain boundary phase diagrams reveals segregation transition in Al-Cu and Al-Cu-Mg alloys. Comput Mater Sci 2021;199:110717.

[87] Zhou Xuyang, Wei Ye, Kühbach Markus, Zhao Huan, Vogel Florian, Darvishi Kamachali Reza, et al. Revealing in-plane grain boundary composition features through machine learning from atom probe tomography data. Acta Mater 2022;226:117633.

[88] Guillén-González Francisco, Tierra Giordano. Second order schemes and time-step adaptivity for Allen–Cahn and Cahn–Hilliard models. Comput Math Appl 2014;68(8):821–46. http://dx.doi.org/10.1016/j.camwa.2014.07.014.

[89] Wang Jingying, Cui Chen, Weng Zhifeng, Zhai Shuying. A high order accurate numerical algorithm for the space-fractional swift-hohenberg equation. Comput Math Appl 2023;139:216–23. http://dx.doi.org/10.1016/j.camwa.2022.09.014.

[90] Chuang Pi-Yueh, Barba Lorena A. Experience report of physics-informed neural networks in fluid simulations: pitfalls and frustration. 2022, arXiv:2205.14249, [physics.flu-dyn].